

中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

题 目： 面向多核 NPU 的语义一致结构化切分与流水调度

摘 要：

多核神经网络处理器的计算图切分同时改变并行度、数据搬运、存储驻留和流水线就绪关系。切分后看似均衡的计算负载，仍可能被子图屏障、共享带宽或缓存占用抵消。针对赛题给定的三种硬件语义，本文以统一的结构化求解框架为主线，将原始计算图、可提交方案、实际执行事件和评价指标分层建模，在统一合法性约束下构造少量具有明确结构依据的候选，再用与官方结果对应的评价闭环确定输出。

针对问题一，建立包含子图依赖和同核执行顺序的增广任务图，以屏障代价约束切分粒度。算法依据连通结构、重组件及分叉前沿构造候选，并在满足波次秩条件的区域中采用“独立双支路—汇合尾部”三片援助，增加并行机会而不引入依赖环。对问题二，将同核子图合并执行带来的复用写成按触及 Core 数计费的超图模型，结合链与汇合感知的空隙插入、带锚点的两核局部最小割及负载守卫细化，最多评价三个去重完整方案。对问题三，利用私有归约树的结构，证明按子树内部峰值与保留结果之差排序，在非交错的局部存储模型中最小化内部存活前沿；将该次序作为受限候选，并以必要执行下界剪枝、官方 Makespan 严格改进准则接受。

在全部 100 个官方计算图、每图 1~5 核的固定配置下，三种固定求解器均得到 500 个合法结果。问题一和问题二在 2~5 核上的平均加速比分别为 1.9516、2.7514、3.4786、4.0553 和 2.3167、3.2356、3.9711、4.5498。问题三在 5 核上的平均基线加速比为 4.7576；对相同计划进行无 L2 与只读 Cache 配对评价，逐图 Cache 加速比的均值为 1.0829，按字节计算的命中率均值为 31.36%。问题一 5 核求解墙钟均值为 3.522 s，问题二全部 500 格均值为 4.029 s；二者均包含在线构造与选优，属于共享主机上的实测结果。

研究表明，依赖结构、数据复用和物理执行次序需要联合处理，单独减少搬运量或提高 Cache 命中率不能保证完成时间缩短。本文以实际计算图与方案的对应关系解释切分机制，以核心和 Pipe 时间线检验重叠效果，并保留固定构造的负结果及证明适用边界。所给算法提供了可复现的结构化求解方法，不依赖逐用例历史赢家拼接，也不将局部模型最优性外推为一般多核调度的全局最优性。

关键词： 多核 NPU；计算图切分；流水调度；超图复用；归约森林；语义一致性

1 问题重述与分析

1.1 问题背景

神经网络计算图通常同时含有矩阵计算、向量运算和数据搬运。一个 NPU 核心内的 Cube、Vector、搬入和搬出单元可在满足依赖时并行工作；多个核心又共享核外 DDR。因此，计算图上的一条边不仅表示逻辑先后关系，还可能在切分后转化为存储分配、数据复制与跨核同步。图切分的效果需要在最终执行过程中判断。

传统多处理器调度中的工作量平衡和关键路径分析提供了基础工具，但资源增加或局部次序变化可能引起调度反常现象 [2]。本题进一步包含有限的 L1 / UB、固定核内调度规则、共享 DDR 和只读 FIFO Cache。一个子图的执行时间会随其所处计划和其他核心的活动改变，因而不能简单地给每个子图赋一个固定标量权重，再把问题化为普通的独立任务装箱。

本文遵循赛题规定 [1]，以未修改评价程序在固定配置下计算的总完成时间 **Makespan** 为主要质量指标，同时报告额外 DDR 搬运量，问题三另报告按字节统计的 **Cache** 命中率。生成方案的程序墙钟是另一类指标。模拟周期衡量方案，墙钟衡量求解程序，两者分别评价。

1.2 输入、输出及三问的关系

输入是 Op-Tensor 二部有向无环图。张量结点携带位置及字节数，操作结点携带类型、所属 Pipe 及周期数；依赖边连接张量和操作。输出由两个字段构成：**node_to_subgraph** 将全部非 COPY 操作映射到子图编号；**core_schedules** 为每个 Core 列出子图的执行顺序。算法可以决定切分、分核及子图顺序，不能直接修改官方核内调度程序、原始计算量、缓存容量或带宽。

三问共享这个输出接口，但对方案的解释不同。

比较对象	问题一	问题二	问题三
执行 Task	一个子图一个 Task	同核全部子图合并为一个 Task	同问题二
同核跨子图数据	经过 DDR，Task 间不保留	可在核内驻留复用	同问题二
跨核同步	前驱 Task 完成后等待	对应 COPY_OUT 完成后等待	同问题二，命中也不取消依赖
核外只读 Cache	无	无	所有核心共享，FIFO 淘汰
主要新增决策影响	切分粒度与屏障	复用、容量和发射次序	查询次序、命中与双带宽池竞争

问题一要求降低切分造成的启动、切换和边界复制代价，同时保持足够并行度。问题二放开同核驻留能力，原来每个子图独立的容量压力转变为跨子图的存活区间压力，分核与排序必须联合考虑。问题三进一步允许跨核读共享，其收益取决于请求是否在正确时间到达、数据是否仍在 Cache 以及其他请求的竞争状态。本文将这一递进关系贯穿模型、算法和图解，而不是分别构建三个互不关联的评分公式。

1.3 难点分解与研究路径

第一，逻辑图合法不等于分核次序合法。把操作收缩为子图后，需要重新检查子图间有向环；加入各核的执行顺序后，还可能形成新的等待环。合法性必须在完整提交方案上

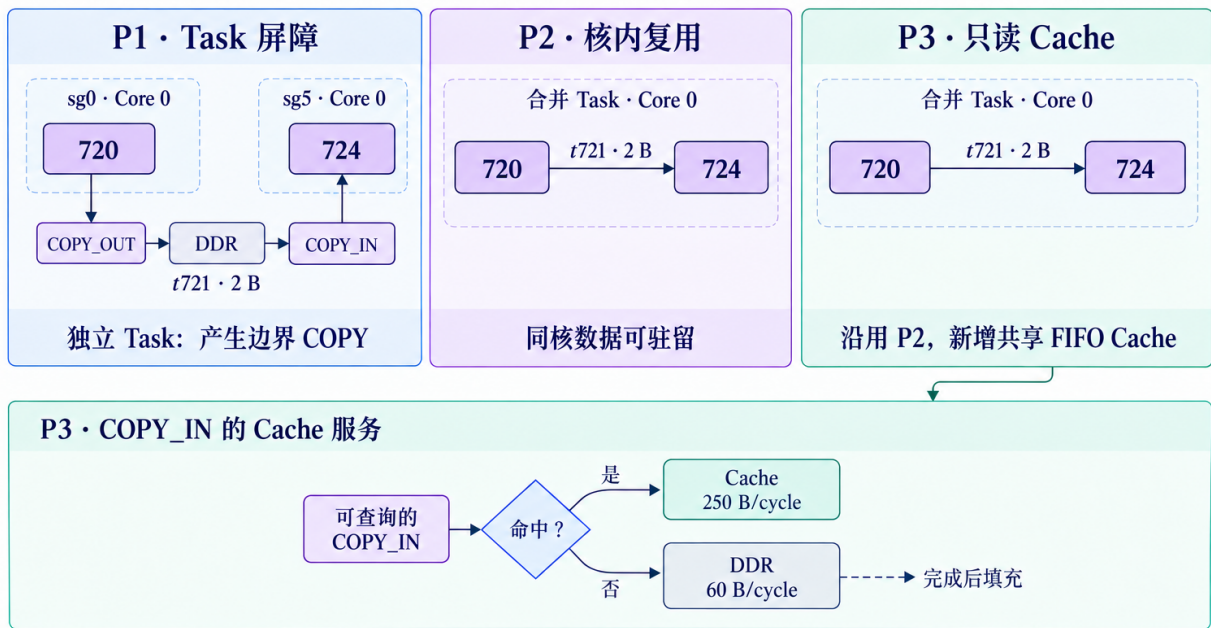


图 1.1 三问执行语义的递进。相同图片段和分核方式下，P1 的独立 Task 边界需要 COPY；P2 允许核内驻留；P3 为可查询的 COPY_IN 增加共享只读 Cache。

判定，不能只看每一块内部连通或每一条原始依赖。

第二，边界代价具有复用和时序效应。同一个输入被多个操作消费时，数据搬运次数与消费者触及的 Core 集合有关，并不总是与切断的操作依赖边数成正比。与此同时，保留数据占用缓存，消除一次复制也可能延长存活区间，挤出其他数据。

第三，真实流水重叠由动态资源共同决定。计算单元空闲可能源于数据尚未到达、跨核释放未到、存储申请次序未轮到或其他执行前驱。仅凭时间线上的白色间隙不能断言其原因；需要把图、编译事件和 trace 关联起来。

第四，复杂模型必须与求解成本相匹配。题面建议避免暴力迭代搜索，以单个用例 5~10 min 给出结果为佳。本文采用结构识别、局部精确子问题与有界候选组合，使每一次评价都有明确用途；不把“限定搜索次数”本身作为方法合理性的论据。

由此形成全文的研究路径：先明确可提交自由度与执行语义；再提取适合构造候选的结构量；用合法性和有适用域的必要下界排除无效候选；最后在完整方案上评价、选择并保留可复核结果。三问共用该逻辑，分别引入屏障、复用和缓存次序三个层次的机制。

2 模型假设与符号说明

2.1 依据题面采用的条件

1. 原始图及官方配置保持不变。张量和操作 ID 在同一图内唯一，字节数及周期字段非负，原始依赖图无环；不删除计算、不改变操作数值语义。
2. 核心同构，每核 M、V、MTE2、MTE3 各有单槽执行资源。同核不同 Pipe 与不同 Core 可并发，但发射仍须满足依赖、Task 激活和存储申请条件。
3. 核内 L1 与 UB 容量分别为 524288 B 和 131072 B，两个存储池独立检查。所有 DDR 读写共享 60 B/cycle，问题三 Cache 容量为 1048576 B，命中读共享独立的 250 B/cycle 带宽池。
4. 问题一的同核 Task 切换等待为 100 cycles，跨核前驱 Task 等待为 1000 cycles；问题二、

- 三跨核 COPY 同步等待为 500 cycles。具体等待作用于题面规定的完成与释放事件。
5. 官方核内调度、换入换出、存储申请顺序、整数周期及事件处理规则均属于执行语义。最终结果以原评价程序为准，不以理想并行模型替换。

以上是题目条件的建模化表达，不额外假设带宽无限、Cache 完美、计算与搬运必然完全重叠或所有任务耗时与环境无关。后文树排序、超图计费 and 候选下界所需的附加条件均在命题处单独给出；结构识别不通过时不使用相应结论。

2.2 对象层次与术语约定

为避免同一个“图”字承载不同约束，全文区分下列对象。

层次	对象	保留的信息	不能据此直接声称
输入层	原始二部图	操作、张量、大小、位置及原依赖	已经有执行次序
计算投影层	计算依赖 DAG	保守保留的原计算因果	完整 COPY/MEM 关系
决策层	子图、分核及核序	参赛者能提交的两字段方案	直接控制操作发射时刻
结构层	增广子图 DAG	数据依赖与全部核链	容量及动态执行成功
编译层	Task 与扩展事件图	COPY、spill、FIFO、存储申请规则	所有事件具有环境无关的时长
运行层	事件轨迹	发射、完成、驻留、共享服务与 Cache	单一聚合分数足以恢复整个轨迹
观测层	M、DDR、命中率及墙钟	各自明确定义的结果	不同指标可以相互替代

“子图”是方案中的划分对象；“Task”是评价器按场景构造的执行对象，二者只在问题一中一一对应。“原始张量”“编译后物理存储实例”和“Cache 查询使用的逻辑标识”分别按所在层解释，不能仅因显示名称相同就视作同一内存对象。

用 $\Theta = (G, q, K, C, H, R, I)$ 标记一次评价的语义身份： C 为配置， H 为官方源码身份， R 为相关运行环境， I 为入口、解析及调用契约。性能环境另记录硬件和并发。固定源码不能单独证明跨解释器或浮点运行时完全一致。

形式合法的输入集合、增广无环的结构方案集合和官方成功执行的方案集合分开记为 $\mathcal{P}_{\text{in}}$ 、 $\mathcal{P}_{\text{ord}}$ 、 $\mathcal{P}_{\text{exec}}$ 。Makespan 只在成功结果上定义。官方拒绝、外部超时、取消、服务错误和未评价均保留各自状态；超时并不证明方案必然无效，也不填成零周期或无穷成绩参加均值。

本文的“精确”必须带对象：可以是某个费用公式精确、固定局部子问题精确、特定输入域的评分等价，或完整执行结果逐字段一致。四者不能省略限定后统一称为“精确求解”。类似地，“下界”必须说明约束全问题、受限方案族、一个固定方案还是某个理想模型。

2.3 主要符号

符号	含义	单位或范围
$G = (O \cup T, E)$	原始操作—张量二部图	DAG
$V \subset O$	非 COPY 原始操作集合	待划分对象
b_t, ℓ_t	张量 t 的大小及位置	B; DDR/L1/UB
d_v, p_v	操作计算时长及 Pipe	cycles; M/V
K	Core 数	$1, \dots, 5$
$\pi(v)$	操作所属子图	非负整数
$a(s)$	子图所属核心	$0, \dots, K - 1$
σ_k	第 k 核的子图序列	有序列表
$P = (\pi, a, \sigma)$	完整可提交方案	两字段 JSON 的数学表示
$\mathcal{C}_q(G, P)$	场景 q 的 Task 与事件编译	含资源和同步约束
$M_q(P)$	官方 Makespan	cycles
$D_q(P)$	官方额外 DDR 搬运量	B
B_i	第 i 图官方单核启发式基线	cycles
$S_{q,i,K}$	第 i 图的基线加速比	$B_i/M_q(P_{i,K})$
$G_{i,K}$	P3 同计划 Cache 加速比	$M_2(P_{i,K})/M_3(P_{i,K})$
$H_{i,K}$	P3 按字节计算的 Cache 命中率	$[0, 1]$
L, U	同一优化域的必要下界与可行上界	cycles
T_{solve}	读入至方案落盘的求解墙钟	s

后文出现 $d_v = \max(1, \text{cycles}_v)$ 时，专指与官方计算投影一致的正整数执行时长；输入的原始字段仍允许为零。图中张量用圆角小框，操作用矩形，子图用带编号外框，核心用水平泳道。该约定用于区分“数据是什么”“计算做什么”“谁先谁后”和“放在哪个 Core”四种信息。

2.4 指标定义与比较边界

对每个核数，平均加速比定义为

$$\bar{S}_{q,K} = \frac{1}{100} \sum_{i=1}^{100} \frac{B_i}{M_q(P_{i,K})}.$$

问题一、二的题目主曲线在 $K = 1$ 处按单核基线定义为 1；算法输出的优化单核方案可以另报 $B_i/M_q(P_{i,1})$ ，不得把二者混写。由于 B_i 是启发式基线而非单核最优值，加速比大于 Core 数不自动构成异常。

问题三的 Cache 比较使用同一个 $P_{i,K}$ 、图与配置，分别启用和关闭 L2。正文同时给出 $\text{mean}(B/M_2)$ 、 $\text{mean}(B/M_3)$ 与 $\text{mean}(M_2/M_3)$ ；前两条均值曲线之商通常不等于第三个量。

额外搬运量采用官方结果中的“调度后 COPY 字节数减去原始图 COPY 字节数”。问题三的该字段仍沿用官方 COPY 计费口径，不因命中而任意改计为零；因此不能把它直接称为 Cache 之后的实际物理 DDR 流量。Cache 命中率使用命中字节除以查询字节；跨用例给出的均值为逐图命中率的算术平均，不是把所有图字节拼接后得到的流量加权命中率。

求解墙钟包含输入读取、预处理、候选生成、在线评分、回退和方案落盘；在外部完成的最终官方复评单独报告。在线使用官方评价的时间已经属于求解墙钟，不能再次从中剔除。共享主机的并发批次墙钟、逐格进程时间和独占机器延迟是不同量，本文不相互替

代。

3 统一模型与本文研究框架

3.1 从计算图到执行事件的分层模型

设 V 为全部非 COPY 原始操作。切分映射 $\pi: V \rightarrow \mathcal{S}$ 的非空原像形成子图集合，分核函数 $a: \mathcal{S} \rightarrow \{0, \dots, K-1\}$ 与序列 σ_k 满足：每个子图恰出现在一个序列中，且恰出现一次。由原始张量依赖导出子图关系 E_S ，再加入同核序列中相邻子图之间的顺序边，得到

$$D^+(P) = (\mathcal{S}, E_S \cup E_\sigma).$$

提交方案至少须满足覆盖、唯一调度与增广图无环。合法方案在场景 q 下经官方规则生成 Task、边界 COPY、换入换出和同步事件，再由事件模拟器得到结果。将该过程写为

$$P \xrightarrow{C_q} \mathcal{E}_q \xrightarrow{\mathcal{R}_q} (M_q, D_q, H_q).$$

这里 $\mathcal{E}_q$ 不只是一个固定权重的 DAG：共享 DDR 中操作的剩余工作量会随并发请求数改变，Cache 状态又由发射、完成与淘汰事件更新。把它写成“编译—执行”两层，是为了区分静态依赖和动态资源竞争，并未假设可以在编译时精确确定全部事件时刻。

优化目标是

$$\min_{P \in \mathcal{F}_q(G, K)} M_q(P), \quad \text{并报告 } D_q(P) \text{ 及问题三的 } H_q(P).$$

其中 $\mathcal{F}_q$ 还受真实容量和事件合法性约束。本文部分选择器用 (M, D) 字典序决策：优先降低 Makespan，时间相同再比较搬运量。这是使候选选择确定化的算法规则，不是题面给出的加权评分公式。P3 Forest 采用更严格的时间改善接受条件，时间持平时保留当前方案。

3.2 以一致语义连接模型、构造和验证

本文围绕三问逐步扩展的执行语义建立统一研究框架：在同一个输入和执行契约下，选择能够保留当前决策所需信息的结构表示，并明确哪些简化只支持构造、哪些结论足以支持拒绝、哪些收益仍须完整评价。它落实为三项要求。

第一，表示的精细程度随决策而变化。判断分叉独立性使用计算依赖图；估计复用使用张量超边及触及 Core 集合；安排归约顺序使用子树内部峰值和根结果大小。每种表示都有使用域，不能把一个只描述字节数的状态当作足以预测未来 FIFO、Cache 和 DDR 行为的完整机器状态。

第二，机制可以并行组合。问题一的组件装箱与分叉援助、问题二的基线与 gap 候选、问题三的现有构造与森林排序，都可以从同一输入产生竞争方案。总体框架因此是具有守卫与回退的机制 DAG，而不是所有样本都必须经过同一串变换的长链。

第三，候选构造与接受证据分开。启发式可以提出可能更好的方案；合法性证书只能说明不会因相应约束失效；必要下界可以在限定域中拒绝不可能胜出的候选；最终质量由完整执行确定。这样可以同时使用简洁的结构模型和忠于物理语义的评价，不要求每一步都重建整个模拟器。

3.3 增广无环与局部重构的安全边界

命题 3.1 (秩函数证书)。若存在整数函数 $r: \mathcal{S} \rightarrow \mathbb{Z}$ ，使增广任务图每条边 $s \rightarrow t$ 都满足 $r(s) < r(t)$ ，则 $D^+(P)$ 无有向环。

证明。若有环 $s_1 \rightarrow \dots \rightarrow s_m \rightarrow s_1$ ，沿环严格递增给出 $r(s_1) < r(s_1)$ ，矛盾。□

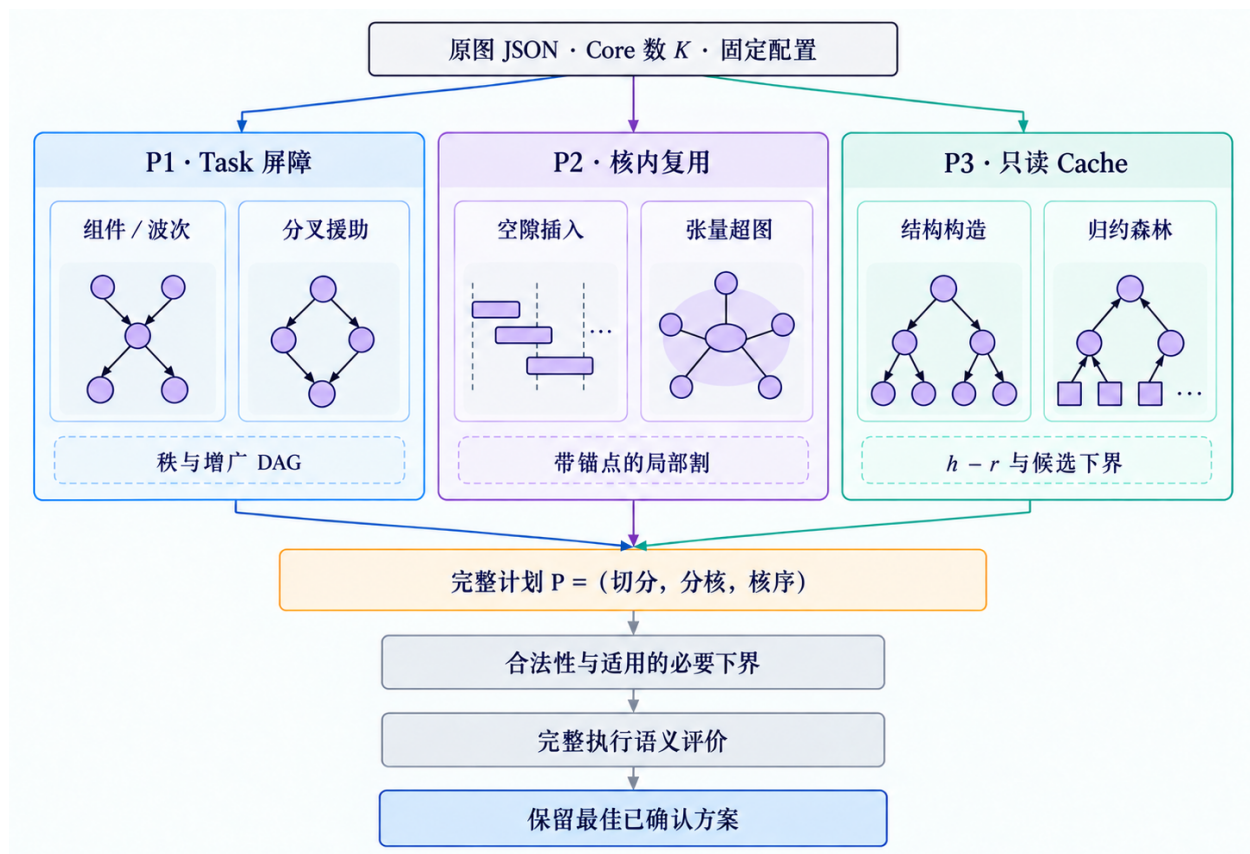


图 3.1 本文研究框架。三问各自根据结构提出候选，共同通过合法性、适用的必要界与完整执行评价决定接受；各机制按适用条件启用。

该命题说明局部构造可通过保持外部秩、为内部新子图细化秩来证明安全；它不说明方案具有容量可行性或更短执行时间。实际算法仍进行完整图检查。

一个容易误用的替代条件是“每个子图路径闭合”。即使两个子图内部各自没有绕出再返回的路径，也可能存在两条原始依赖 $a_1 \rightarrow b_1$ 和 $b_2 \rightarrow a_2$ ；将 a_1, a_2 分在一组、 b_1, b_2 分在另一组后，商图出现双向边。原图无环而商图有环，因此局部闭合检查不能替代完整增广无环检查。此反例也解释了为什么切图可视化必须同时展示跨边界依赖，而不能只画互不相交的彩色框。

3.4 必要下界、可行上界与局部最优性

对保留全部原始计算、满足每核每 Pipe 单槽条件的执行，令

$$W_M = \sum_{v:p_v=M} d_v, \quad W_V = \sum_{v:p_v=V} d_v.$$

在确认原始计算依赖被真实执行保留的投影图上，设 L_{cp} 为最长计算路径，则有基础必要下界

$$L_{\text{basic}} = \max \left\{ L_{cp}, \left\lceil \frac{W_M}{K} \right\rceil, \left\lceil \frac{W_V}{K} \right\rceil, \max_v d_v \right\} \leq M.$$

工作量项分别针对 M、V 计算。两条 Pipe 可并行，因此不能将 $W_M + W_V$ 作为同一服务器工作量直接除以 K 。投影也不能任意收缩经过 COPY 的路径，再把由此得到的关系解释为原始计算的直接边。

对于固定候选 P ，若能额外确定同核同 Pipe FIFO 和必要跨核释放关系，可得到更强的候选下界 $L(P)$ 。当当前成功方案的完成时间为 U ，且 $L(P) \geq U$ ，即可拒绝严格改善不可能的这个候选。这里 $L(P)$ 约束的是已固定的候选，通常不是所有方案最优值的下界。把候选剪枝界拿来评估全问题最优差距，会混淆优化域。

若 $L \leq \text{OPT} \leq U$ 确实来自同一优化域，则

$$\frac{U}{\text{OPT}} - 1 \leq \frac{U}{L} - 1.$$

这给出保守的质量保证。 $U/L - 1$ 小可以证明距离最优较近，值大则可能来自算法不足，也可能来自下界放松，不能被解释为一定可实现的改进空间。

3.5 状态压缩为什么需要更强的证明

上述统一建模进一步提出一个问题：能否用比完整方案更小的状态，正确决定后续修改及其结果？这一问题有助于限定编译复用和局部响应模型的解释范围，但当前三种主算法并未实现一般的最小未来充分状态。

先固定允许的动作集合 $\mathcal{A}$ 及每个动作的参数语义，例如在一个声明的结构域内切分、合并或移动子图；动作 a 的定义域为 D_a 。选定在所声明研究域上有定义的观测 Φ ：在成功执行域可以只含 **Makespan**；若域含未成功状态，则另用明确的结果状态观察，不能把未定义的 M 填为数值。不同输出投影对应不同等价关系。令 $L(P)$ 是从 P 出发可执行的有限动作词集合，包含空字。未来行为等价要求

$$L(P) = L(Q), \quad \Phi(w(P)) = \Phi(w(Q)), \quad \forall w \in L(P).$$

仅比较双方“碰巧都合法”的动作，会漏掉可用动作不同的情况；仅比较当前 $M(P) = M(Q)$ ，也不能排除下一次切分后 FIFO 或存储行为不同。因此当前分数相同并不自动产生可以安全搜索的商空间。

命题 3.2 (受限域的未来充分表示)。给定压缩 S , 若对每个动作存在可用性判定 d_a 、状态更新 $\bar{a}$ 及当前观测函数 o , 并且对声明域内全部方案满足

$$P \in D_a \iff d_a(S(P)), \quad S(a(P)) = \bar{a}(S(P)), \quad \Phi(P) = o(S(P)),$$

则 $S(P) = S(Q)$ 蕴含二者未来行为等价。

证明。长度为零的词由观测条件成立。假定任意长度 j 的共同可执行词作用后压缩状态仍相等, 则可用性条件保证下一动作在两侧同时可用或同时不可用; 更新条件使执行后的压缩状态相等, 再由观测条件得到输出一致。对词长归纳, 合法语言与全部未来观测相同。□

该命题给出的是充分条件。要称为“最小”, 还需证明不同压缩状态能够被某个未来词区分, 或证明所得划分最粗; 把完整方案编码进一个整数并未减少信息成本。本文对实际优化只使用已经核定的较弱对象: 完全相同计划去重、满足键条件的局部编译复用、受限模型的响应表示和局部候选构造。数学深度体现在区分这些保证并证明需要的条件, 不依靠把所有复用统称为低维商。

3.6 有界候选与求解复杂度的组织

统一求解流程为: 解析与结构识别; 构造基准方案; 按输入结构产生有限候选; 去重并检查合法性; 使用适用的必要条件剪枝; 评价剩余完整方案; 输出最佳已确认方案。算法没有用例编号到赢家的查表分支, 历史实验只用于设计固定规则。

设解析和结构构造总成本为 $C(G, K)$, 完整图合法性检查为 $O(|V| + |E_V| + |\mathcal{S}|)$, 实际评价次数为 b , 各次评价成本为 T_j , 则总体成本可记为

$$T_{\text{solve}} = O(C(G, K) + b(|V| + |E_V| + |\mathcal{S}|)) + \sum_{j=1}^b T_j + T_{\text{IO}}.$$

该表达式不把评价器时间隐藏在一个常数中。本文同时给出候选数量上限、实际调用账和端到端墙钟; 对于内部精确最小割等子过程, 再在对应章节说明其规模及调用上界。有限候选保证评价次数有界, 求解高效还依赖候选规模、结构识别成本及资源上限。

4 问题一: Task 屏障感知的结构化切分

4.1 从负载均衡转向边界与并行度的联合选择

问题一中, 每个子图成为独立 Task。若将一个大于图拆成两个小子图, 可能把工作交给空闲核心, 也会产生边界数据搬运和新的激活条件。即使两个子图位于同一 Core, 其间数据仍经 DDR, 因而不能以“同核切分没有通信成本”作为模型假设。

设 Task s 在 Core k 上开始于 A_s 、完成于 F_s , 其同核前一个 Task 为 $\text{prev}(s)$ 。激活时刻至少满足

$$A_s \geq F_{\text{prev}(s)} + 100,$$

$$A_s \geq F_t + 1000, \quad t \in \text{pred}(s), \quad a(t) \neq a(s).$$

不存在的前项不参与最大值。这里 $F_s - A_s$ 仍受其他核的 DDR 活动影响, 不是只由子图结点周期相加得到的固定时长。细粒度切分增加可并行结构, 粗粒度切分减少屏障和复制, 两者之间的取舍构成问题一的核心。

算法首先保留依赖紧密、复用较强的局部结构, 再为负载过重或前沿并行度不足的部分构造补充方案。这样的切分有明确对象: 完整组件装箱解决独立工作分配; 重组件和汇

合尾部处理解决单一大块拖尾；分叉前沿与波内援助解决尚未暴露的并行分支。构造规则运行前冻结，由图结构触发，不从历史成绩表挑选方案。

4.2 固定候选族及适用性守卫

基础构造最多产生六种去重候选，包括组件有界装箱、重组件或汇合尾处理、过载分解、共享输入处理、分叉前沿和容量受限的私有返回链构造。共享输入规则考虑外部输入字节及组件关系；返回链规则先识别私有的 $M \rightarrow V^+ \rightarrow M$ 形状及容量条件。结构不满足时保留既有候选，不强行将一般 DAG 解释成链或树。

基础候选之上有三类有守卫的细化：容量返回赢家的变长分包响应细化；两种保持局部结构完整的前沿次序调整；波内三片援助。它们共享当前输入生成的父方案及其已知评分，以严格改善为接受条件。六个基础候选、至多一个响应细化、至多两个完整前沿候选和至多一个援助候选，使整个固定入口的在线 E1 评分上限为十次；去重、单候选快速路径和不适用分支使实际次数通常更少。

E1 用于在线完整方案比较；最终论文结果由官方 E0 的实际原件确认。响应编译保留核内 FIFO、依赖、存储门控、COPY 工作量、整数周期及共享带宽的关键结构。相同编译签名、共同空状态释放等条件下可以复用部分编译或响应计算，但这里不宣称一个有限签名已对任意未来输入充分，也不宣称有理数实现与官方 binary64 在全域严格等价。

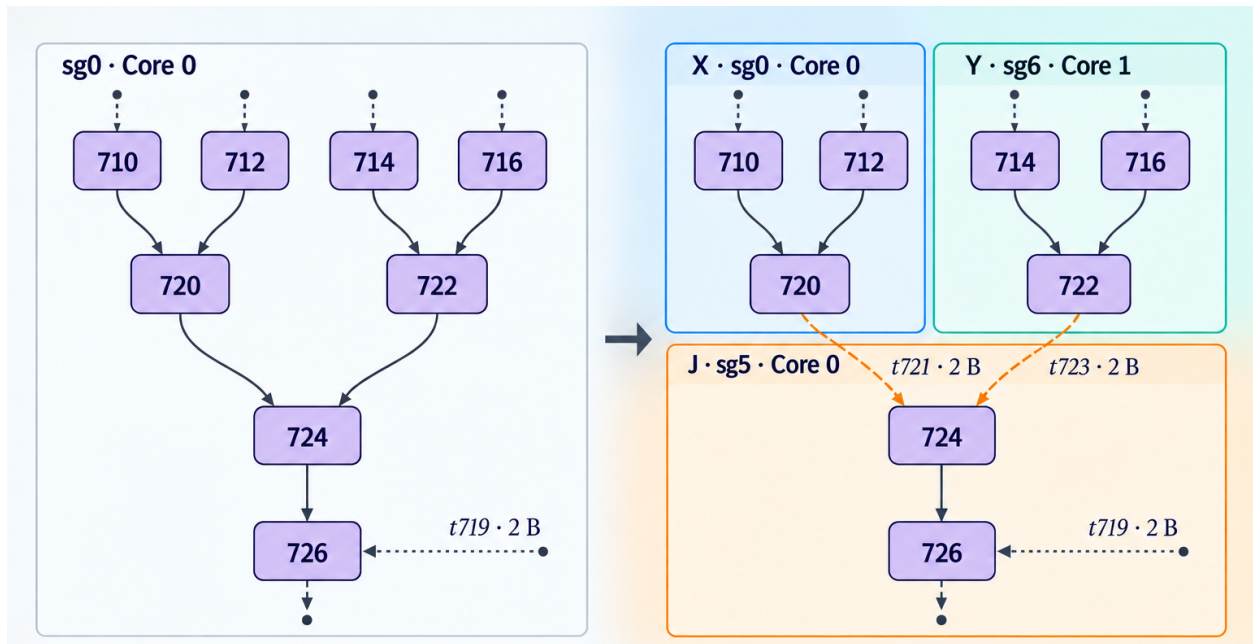


图 4.1 case_026、5 Core 的真实归约片段。左为原始划分，右为分叉援助；八个 ADD 节点的编号和依赖不变。点线端口连接裁剪范围外的输入或后继，橙色虚线为跨子图依赖。

4.3 波内三片援助的构造与无环性

在父方案的子图数据 DAG 中，令 $h(s)$ 为 Task 的最长前驱高度。算法只处理每核顺序严格增加 h 的区域；同一高度、不同 Core 的 Task 之间必须不存在计算依赖。将负载较重核心称为 donor，较轻核心称为 helper，一次构造中二者角色互斥，helper 在同一波中须有可核对的原 Task，而不是任意空核。

在 donor 内选取符合识别条件的分叉—汇合结构，把计算划分为互不重叠的 X, Y, J ：Y 为可导出的独立支路，X 为留在 donor 的其余前部，J 为汇合及后续尾部。要求 X, Y 之间没有依赖，J 不向 X, Y 返回，外部边与原波次方向相容。helper 先执行导入的 Y，再执

行本波原 Task；donor 执行 X 后执行 J 。构造按主导 Pipe 工作量选择负载转移，只有代理最大负载严格下降才继续形成完整候选。

命题 4.1（三片援助的增广无环性）。在上述守卫条件下，并保持不同波按原高度先后排列，三片构造不在增广任务图中引入有向环。

证明。给每个 Task 赋字典序秩 (h, ϕ, j) 。第一项为原波高度；同波内，独立的 X 、 Y 放在前部阶段，helper 接收的多个 Y 按其实际顺序以 j 区分，helper 原 Task 位于其后，汇合尾 J 位于需要的 X, Y 之后。同波不同原 Task 没有计算边， X, Y 之间无边， J 无返回边，donor/helper 角色互斥，故所有新增数据边与同核序列边均沿同波秩向前。跨波边严格增加第一项。由命题 3.1，增广图无环。□

证明只覆盖构造守卫对应的依赖与次序安全。导出支路会增加某些数据边界，新的 FIFO 与存储申请还可能改变换入换出。因此，代理负载下降不能作为接受收益的证据。构造完成后仍检查完整提交方案，并用完整 E1 结果比较 (M , scheduled COPY)。

4.4 算法流程、停止条件与复杂度

算法 1：屏障感知的固定结构候选求解。

输入：原图 G ，核数 K ，冻结配置

- 1 解析图：按固定结构规则构造基础方案族，按规范化计划去重
- 2 若仅一份基础方案则直接保留；否则逐份完整评分，保持最佳已确认方案
- 3 若赢家满足容量返回识别条件，构造一次变长分包细化并比较
- 4 对冻结的两种完整前沿规则，逐一构造、检查、去重并比较
- 5 若父方案满足波次守卫，构造一次 $X/Y/J$ 援助候选
- 6 检查完整覆盖、唯一调度及增广无环；候选严格改善时替换父方案
- 7 输出最佳已确认方案及独立诊断；诊断不写入提交 JSON

解析、拓扑检查及给定候选的覆盖检查与图规模线性相关；排序、组件装箱和前沿优先队列增加相应的排序成本。三片构造还遍历可能的局部见证，不能将整个求解器简单标为一次线性遍历。该援助构造设置 Task 上限 320，援助结构检查的软时间预算为 2 s，附加评分最多一次；软预算通过检查点触发退出，并非每一步都可被精确抢占的硬时限。响应分包具有独立的受限编译预算和子进程停止条件，全部计入求解墙钟。

因完整入口最多十次 E1 评分，其评价成本为 $\sum_{j=1}^b T_j$ 、 $b \leq 10$ 。图越大、存储事件越多，一次评分成本可能越高。实际实验同时保留每格调用数和进程收据，避免用“十次以内”替代实测效率。

4.5 全量结果及质量—代价取舍

固定算法在 100 图、1~5 核的 500 格中全部得到合法结果。题目要求的主曲线见下表；“优化单核”另列，避免改变官方主曲线的单核定义。

核数	1	2	3	4	5
官方平均加速比	1.0000	1.9516	2.7514	3.4786	4.0553
优化方案 $\text{mean}(B/M)$	1.0021	1.9516	2.7514	3.4786	4.0553
改善/持平/退化	0/100/0	8/92/0	15/85/0	12/88/0	13/87/0
求解墙钟均值 / s	0.436	3.316	3.426	3.427	3.522

5 核平均加速比由前版 4.0259 提升到 4.0553，相对增幅为 0.729%。新三片援助在全部 500 格中被选中 36 次；其余格保留父方案。表中的完整版本比较包含固定策略组合，不能把所有改善都归因于三片援助的独立因果贡献。

5 核额外 DDR 总量为 1053684536 B，相比前版增加约 0.389%。因此该次完整版本改进体现了以主要目标换取少量搬运增加的取舍，不能写成时间和 DDR 同时全面改善。5 核墙钟 P95 为 16.623 s，最大值为 25.552 s；该批次共享主机上使用四个 worker，操作系统缓存未清空，不能据此推断独占硬件的确定延迟。

全量调用账包含 500 次新求解进程、1130 次在线 E1，以及 48 次新 E0。另 452 格在原方案字节、图、核数、配置与评价语义逐格核同后复用已有 E0 原件；这些结果仍属于当前固定算法的输出，但旧 E0 墙钟不记作新测值。此区分使结果复用保持可追溯，也避免为了“全部新跑”重复消耗无新增信息的评价。

本章说明，结构化切分可以在保留真实评价闭环的同时，减少盲目枚举。剩余瓶颈可能是不可拆分的计算链、边界同步、共享输入或动态存储行为；因此下一问利用同核驻留后，优化重点需要从“少设 Task 屏障”进一步转向“复用与发射次序共同安排”。

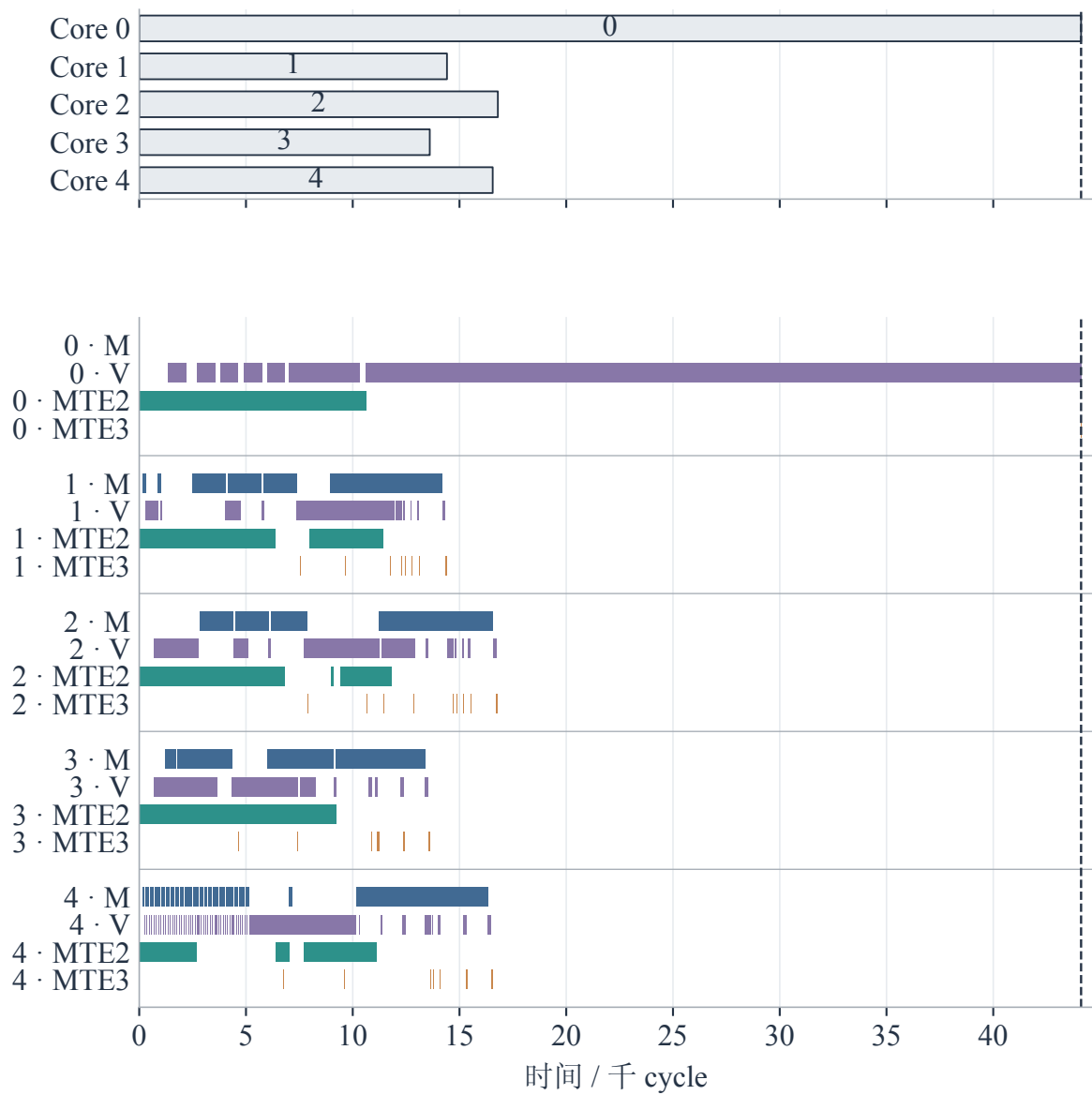


图 4.2 case_026、5 Core 的父方案执行轨迹。上：Task 跨度；下：各 Core 的四条 Pipe。Makespan 为 44,114 cycles，额外搬运 211,680 B。

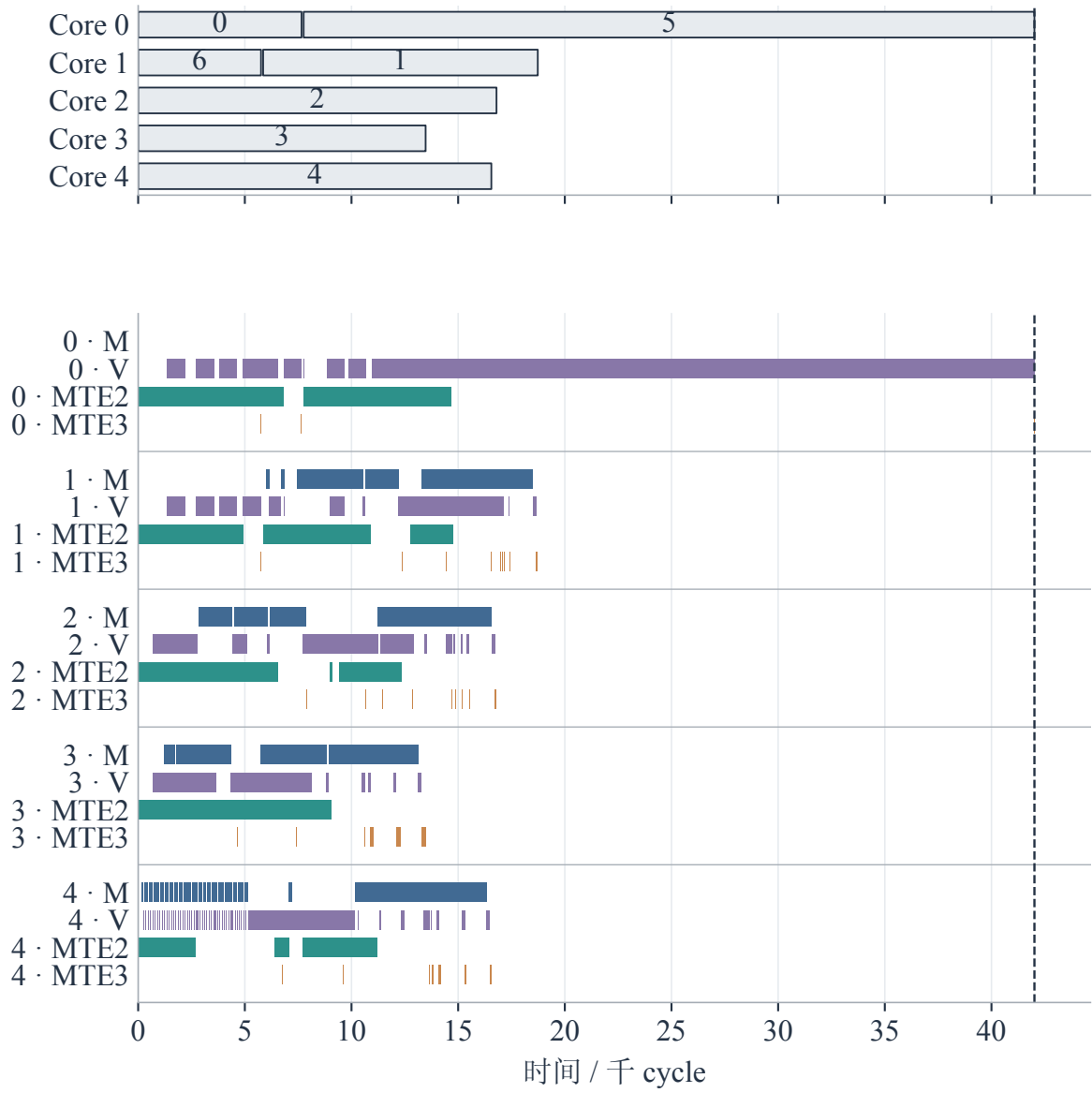


图 4.3 同一用例的分叉援助方案。横轴与父方案一致；Task 数从 5 增为 7，Makespan 降至 42,014 cycles，额外搬运增至 375,532 B。条带均来自官方执行事件。

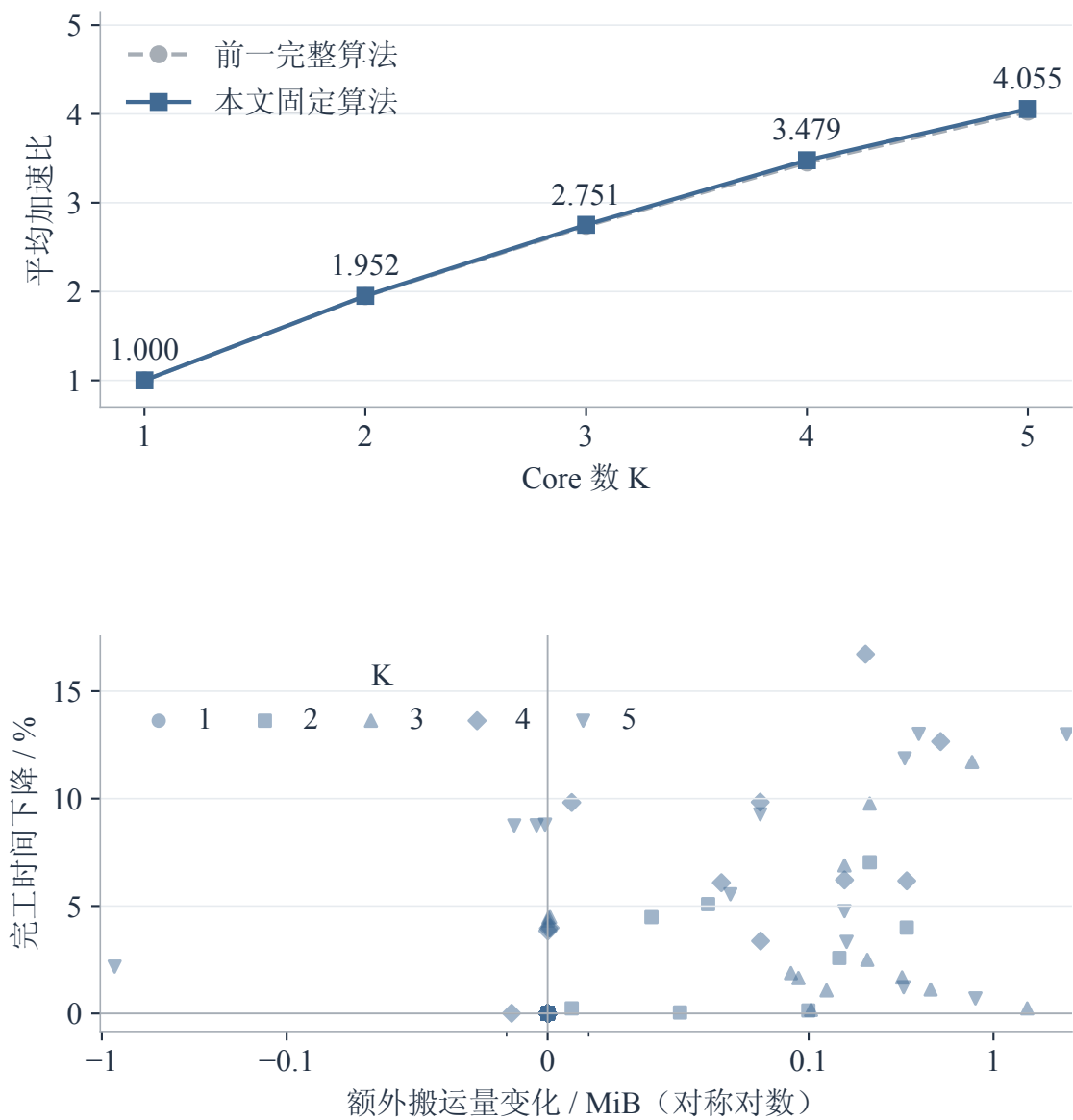


图 4.4 问题一的完整版本比较。上：每个核数的 100 图平均加速比；下：全部 500 格的时间与搬运量配对变化。核数用点形区分，横轴采用对称对数。

5 问题二：复用感知的超图分核与空隙插入

5.1 同核合并改变了什么

问题二把同一 Core 上的子图合并为一个 Task。同核依赖可以保留核内数据，跨核依赖才插入边界 COPY，并在源 COPY_OUT 完成后经过 500 cycles 才允许目标 COPY_IN 进入相应就绪判定。于是，拆成 singleton 子图不再等于为每个操作设置独立 Task 屏障；子图顺序成为影响固定核内调度的重要输入。

这一变化同时带来机会和压力。把共享数据的操作放在同核可减少重复搬运，但数据驻留时间可能延长；按计算负载分散消费者可改善并行度，却扩大触及 Core 集合和跨核同步。本文分别构造计算日历代理与精确的预换入换出 COPY 费用，再由完整方案评价决定二者的最终取舍。

5.2 链与汇合感知的空隙插入

对计算依赖图识别可连续处理的链以及分叉、汇合位置。每个 Core 的 M、V Pipe 维护已安排区间集合，候选操作必须在依赖释放后寻找足够长的空隙。相比仅在某核当前尾部追加工作，空隙插入可以利用前序等待留下的可用窗口。

设操作 v 的代理释放时刻为 r_v ，计算时长为 d_v 。对 Core k 、Pipe p_v 的占用日历 $\mathcal{I}_{k,p_v}$ ，选择最早满足

$$[s_v, s_v + d_v) \cap \mathcal{I}_{k,p_v} = \emptyset, \quad s_v \geq r_v$$

的区间。汇合附近的相关链使用至多 K^2 个核对联比较，使一个支路的局部选择考虑到另一个支路的完成位置。输出以 singleton 子图及其优先序表达，后续仍由官方规则生成实际流水。

这个日历模型故意保持可计算：它不完整模拟共享 DDR、公平分带宽、spill 与 memory-credit。 s_v 是构造代理时刻，不是官方 trace，不是 Makespan 的保守上下界。日历的价值在于产生结构合理的候选，而不是取代官方执行。

同核合并与跨核复制的区别见图 ~1.1。后续分核模型据此区分复用收益与跨核费用。

5.3 按触及 Core 数计费的超图模型

普通图切割按边累加费用，容易对同一张量的多消费者重复计费。超图用一条超边连接同一数据相关的全部操作，以触及 Core 数描述通信，这种建模思想与并行稀疏矩阵计算中的通信建模相通 [3]。本文根据官方边界复制语义构造具体费用。

要求非 COPY 操作集合完整、物理张量不带逻辑别名、每个物理张量至多有一个原生生产者，张量大小为非负整数。对超边 e ，令 $\lambda_e(a)$ 为其端点所触及的不同 Core 数， w_e 为字节权重。则预 Step2 的原始 COPY 费用写成

$$D_{\text{pre2}}(a) = C + \sum_{e \in \mathcal{H}} w_e (\lambda_e(a) - 1).$$

对于核外输入，消费者每增加一个触及的 Core，就增加一次大小为 b_t 的输入复制，故权重为 b_t ；基础的一次输入包含在常数 C 中。对于由计算产生的中间张量，超边包含生产者和消费者，官方边界构造按远端核所需的输出/输入对计费，权重为 $2b_t$ ；最终输出等与分核无关的部分计入 C 。实际实现按官方边界规则处理，不能把所有边一律赋为相同权重。

该公式精确描述适用域内的预 Step2 COPY 字节，不包含容量不足引发的换入换出，也不表达传输之间的重叠或串行。若一组操作 A 从 Core u 移往 v ，只需检查接触 A 的超边。对某条超边，目标核原先无端点时费用增加一个 w_e ；源核全部端点都迁出时费用减少一个 w_e 。由此可增量计算

$$\Delta_e = w_e (\mathbf{1}\{n_{e,v} = 0\} - \mathbf{1}\{n_{e,u} = |e \cap A|\}).$$

这也解释了单结点逐次迁移可能看不到组合收益：第一步迁移增加目标核触及数，只有最后一个源端点迁出时才消除源核费用。局部联合改动比纯逐点贪心更适合保留复用结构。

以一个张量的超边为例，令 u 为生产者， v_1, v_2 为两个消费者，并以 t 标记该张量对应的超边。将两个消费者中的一个迁入生产者所在 Core 后，另一个消费者仍使超边触及两个 Core，费用项暂不下降；只有两个消费者都迁入时， λ_t 才从 2 降为 1，对应费用项从 w_t 降为 0。图中的变化仅针对该超边，不能解释为完整方案的 COPY 字节全部归零。

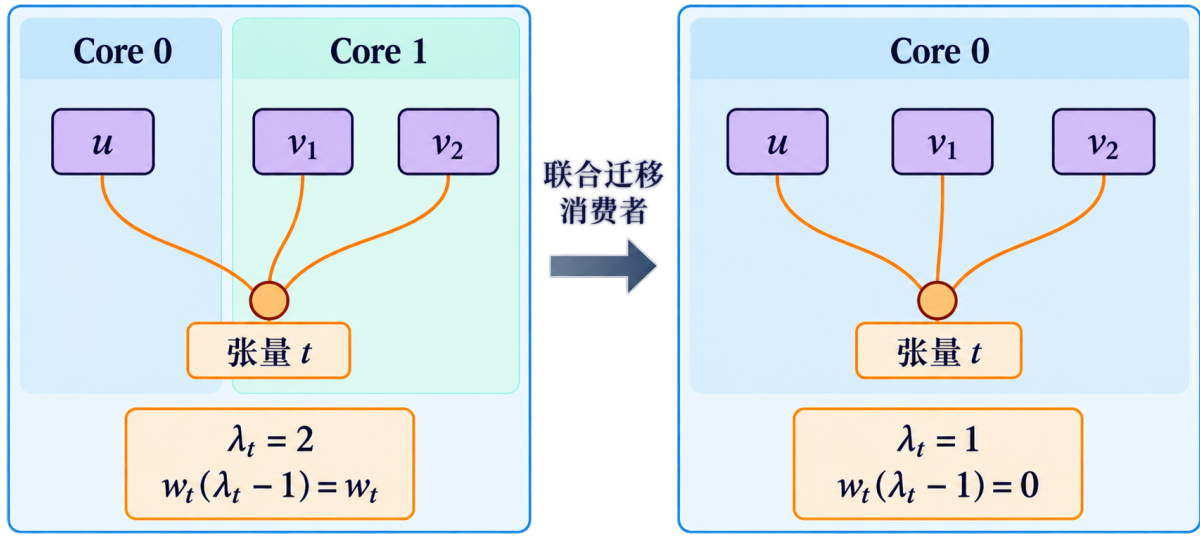


图 5.1 张量超边的触核数与消费者联合迁移示意。左：生产者与消费者分处两个 Core；右：两个消费者均迁入生产者所在 Core。橙色连线表示超边关联。

这一局部字节收益仍需接受执行层的检验：联合迁移可能同时改变计算负载、数据驻留和同步位置，而上述费用没有刻画共享带宽、容量换入换出及流水重叠。下一步因此对局部割加入负载守卫，再用完整方案评价判断 Makespan 是否改善。

5.4 带锚点的两核局部细化与负载守卫

在已构造的 gap 方案上，算法选择核对与最多 16 条链组成的局部区域。区域之外的操作及必须固定的链作为锚点，区域内操作具有两种 Core 标签。对固定区域和锚点，超边“是否同时触及两侧”的费用可通过辅助结点构造二标签最小割问题，得到该局部字节目标的精确解。

但无约束最小割可能将多数工作迁入同一 Core，造成计算负载失衡。实现为各核各 Pipe 设置工作帽；若新分配违反工作帽，逐步将部分迁入链锚回原核后重新求割。原分配始终是可行参考，最多 $r + 1$ 次割 ($r \leq 16$) 后得到满足守卫的接受候选或保留原分配。每次接受还要求字节数不增加。

命题 5.1 (局部费用单调性)。若每轮锚点均保持原分配可行，局部割求得其当前无额外工作帽子问题的精确最优值，且接受前重新核对工作帽，则最终接受方案在该局部预 Step2 COPY 目标上不劣于原分配。

证明。当前锚点约束下原分配可行，因此精确最小割值不大于原分配值。若工作帽不

满足，则增加与原分配相容的锚点，不改变原分配可行性。每轮仍保留这一费用上界。只有同时通过工作帽和实际费用检查的候选才接受，结论成立。□

该过程不等于带所有负载约束的全局精确最小割。新增锚点会缩小可行域，也未纳入容量、共享带宽或真实 Makespan。本文将其称为带守卫的局部字节细化，避免把局部精确性误写为多核调度全局最优。

5.5 完整候选策略与复杂度

算法 2：复用感知的有界完整方案选择。

输入：G、K、冻结配置及完整方案评分接口

- 1 生成结构自适应基线 P0
- 2 构造链/汇合感知 gap 方案 P1；不适用时返回 P0
- 3 在 P1 上做两核、至多 16 链的超图细化
- 4 仅当预 Step2 COPY 严格下降时，对细化分配重新安排次序，形成 P2
- 5 保留未经细化的 P1；按 P0、P1、P2 的构造顺序去重
- 6 对至多三个完整计划在线评分，按 (Makespan, added COPY) 字典序择优
- 7 同分保留先前方案；失败按固定回退规则处理；输出合法两字段 JSON

保留 P_1 很关键：预 Step2 字节没有下降，只能说明不触发细化后重排，不能据此删除原 gap 方案。在线最终选择并不以字节代理代替 Makespan。

设图中有 n 个计算操作、 m 条投影依赖，日历查询累计扫描成本为 Q ；链、汇合识别与相应排序的成本由 n, m 决定。若有 R 个局部细化区域，区域辅助网络规模为 (n_r, m_r) ，一次最小割成本为 $T_{\text{cut}}(n_r, m_r)$ ，则细化总成本不超过

$$\sum_{r=1}^R (w_r + 1) T_{\text{cut}}(n_r, m_r), \quad w_r \leq 16.$$

日历插入及核对比较还包含 Q 与 K^2 因子，因此不将其笼统称为线性算法。最昂贵的完整方案在线评分最多三次，区域局部割不调用完整评价器。各阶段时间均计入求解墙钟。

5.6 完整结果与通信—时间取舍

冻结求解器在全部 500 格上输出合法结果，使用 1131 次原生 E2 评分和 500 次独立 E0 复评，未触发 E0 fallback。主结果为：

核数	1	2	3	4	5
官方平均加速比	1.0000	2.3167	3.2356	3.9711	4.5498
优化方案 $\text{mean}(B/M)$	1.2061	2.3167	3.2356	3.9711	4.5498
改善/持平/退化	59/41/0	64/36/0	54/46/0	49/51/0	42/58/0

相对前一完整版本，500 格共有 268 格 Makespan 改善、232 格持平、0 格退化，涉及至少一个核数改善的图有 69 个。这个结果证明固定组合策略在本组已知官方图上的经验表现，不能证明每个组成机制对所有输入独立保优。

全部 500 格的额外 DDR 总量由 3043860416 B 增至 4159738560 B，增加约 36.66%。其中 55 格时间与搬运均改善，199 格时间改善而搬运增加，14 格时间改善且搬运不变，232

格两者持平。更大的总搬运伴随更短 Makespan，表明仅最小化通信字节不足以描述本题主要目标。

求解墙钟均值为 4.029 s，最近秩定义的 P95 为 18.841 s，最大值为 40.187 s；外部独立 E0 的均值为 1.410 s、P95 为 6.592 s、最大值为 13.980 s。实验使用共享 Mac、单 worker，整批约 46.63 min。此处的“新进程”不意味着清空操作系统磁盘缓存或获得独占算力。

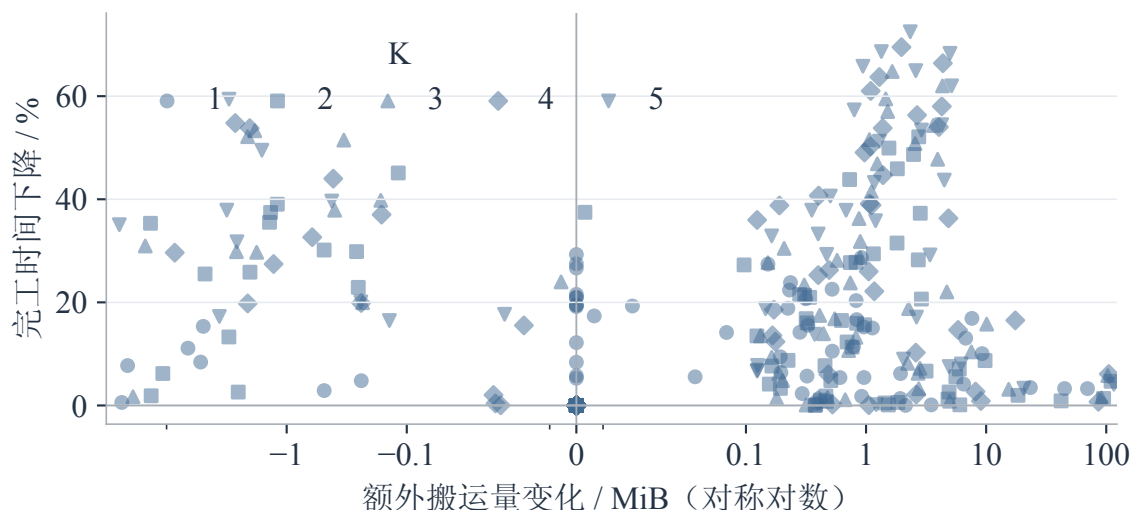
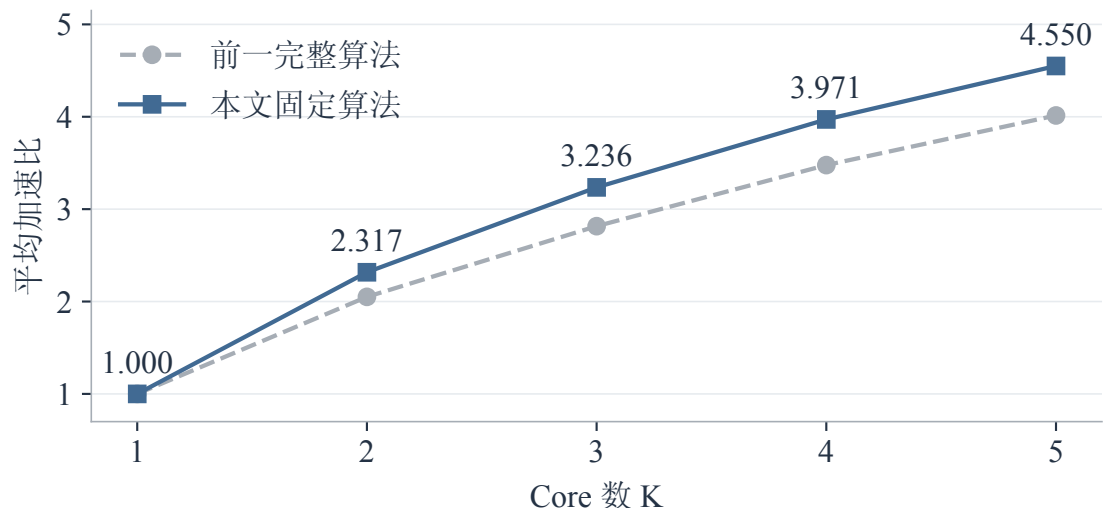


图 5.2 问题二的完整版本比较。上：官方口径的平均加速比；下：500 格配对结果。部分时间改善伴随额外搬运增加，两项指标分别报告。

5.7 负例：零 spill 与少搬运仍不足以保证更快

为检验通信与容量代理的解释力，另一个冻结的零 spill 联合构造 C04 在六个预选 5 核用例上接受官方评价。六格都合法，零 spill 及字节核对成立，但 Makespan 全部劣于本章主算法。

case	主算法 / cycles	C04 / cycles	相对变慢
005	33515	38209	14.01%
016	2091375	2399125	14.72%
019	16247	28514	75.50%
025	942938	975658	3.47%
039	449653	494942	10.07%
075	343930	452120	31.46%

其中 019、039、075 的 COPY 字节下降而完成时间上升。原因不能仅凭这两个聚合量确定；该结果已经足以否定“预估字节降低且零 spill 就必然更快”的推论。计算 FIFO 不变也未必保留 COPY FIFO、共享 DDR 竞争和跨核释放过程。由此，零 spill 证书若只覆盖官方 Step2，不应被宣传为完整执行最优证书。

与问题一相比，本章把同核驻留带来的收益纳入候选构造，同时保留对动态执行的谨慎。下一章再加入共享 Cache 时，复用机会还要从“消费者在哪个 Core”延伸到“消费者何时请求”。

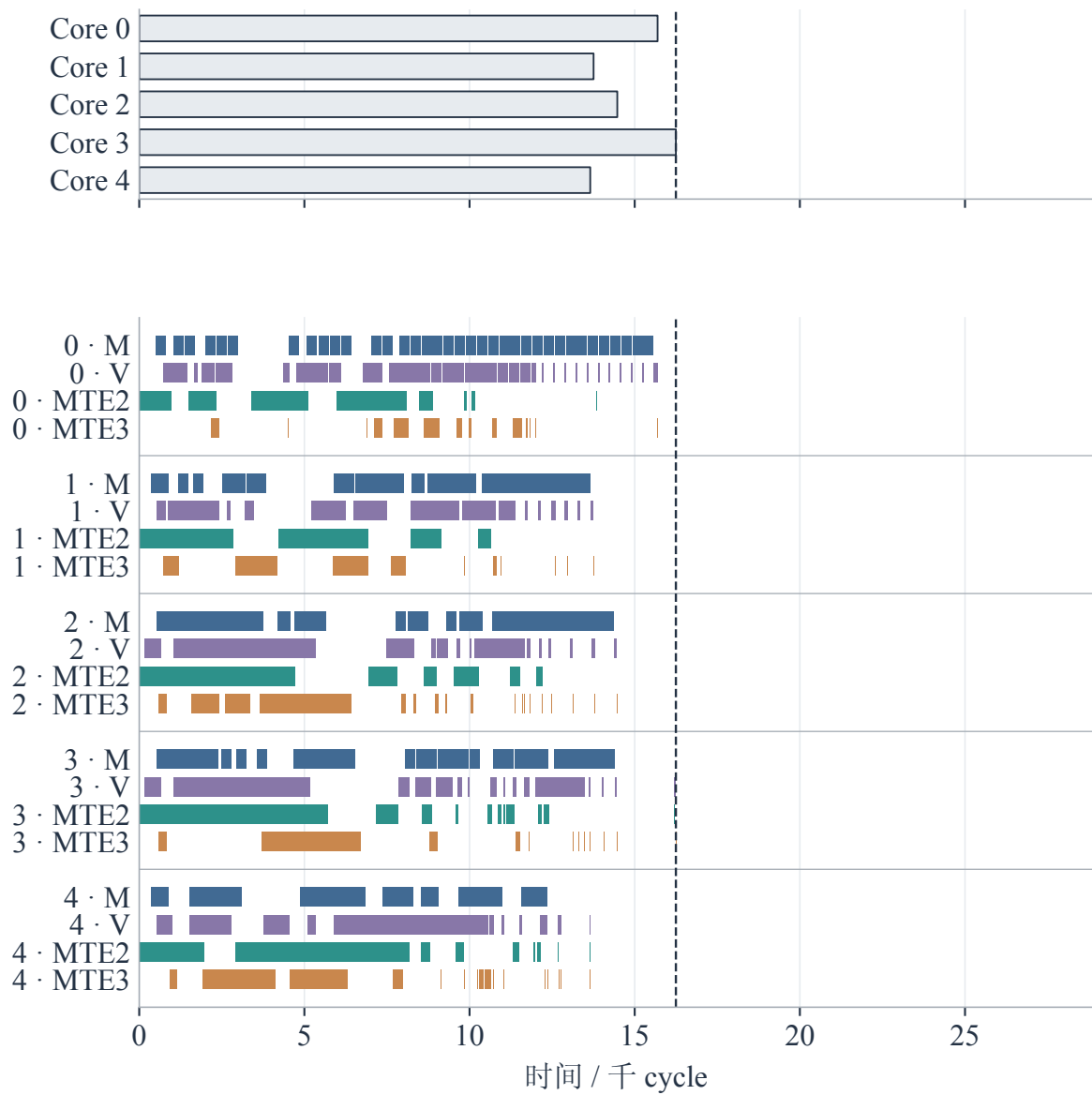


图 5.3 case_019、5 Core 主算法的真实执行时间线。上为合并后的 Task，下为四条 Pipe；Makespan 为 16,247 cycles。

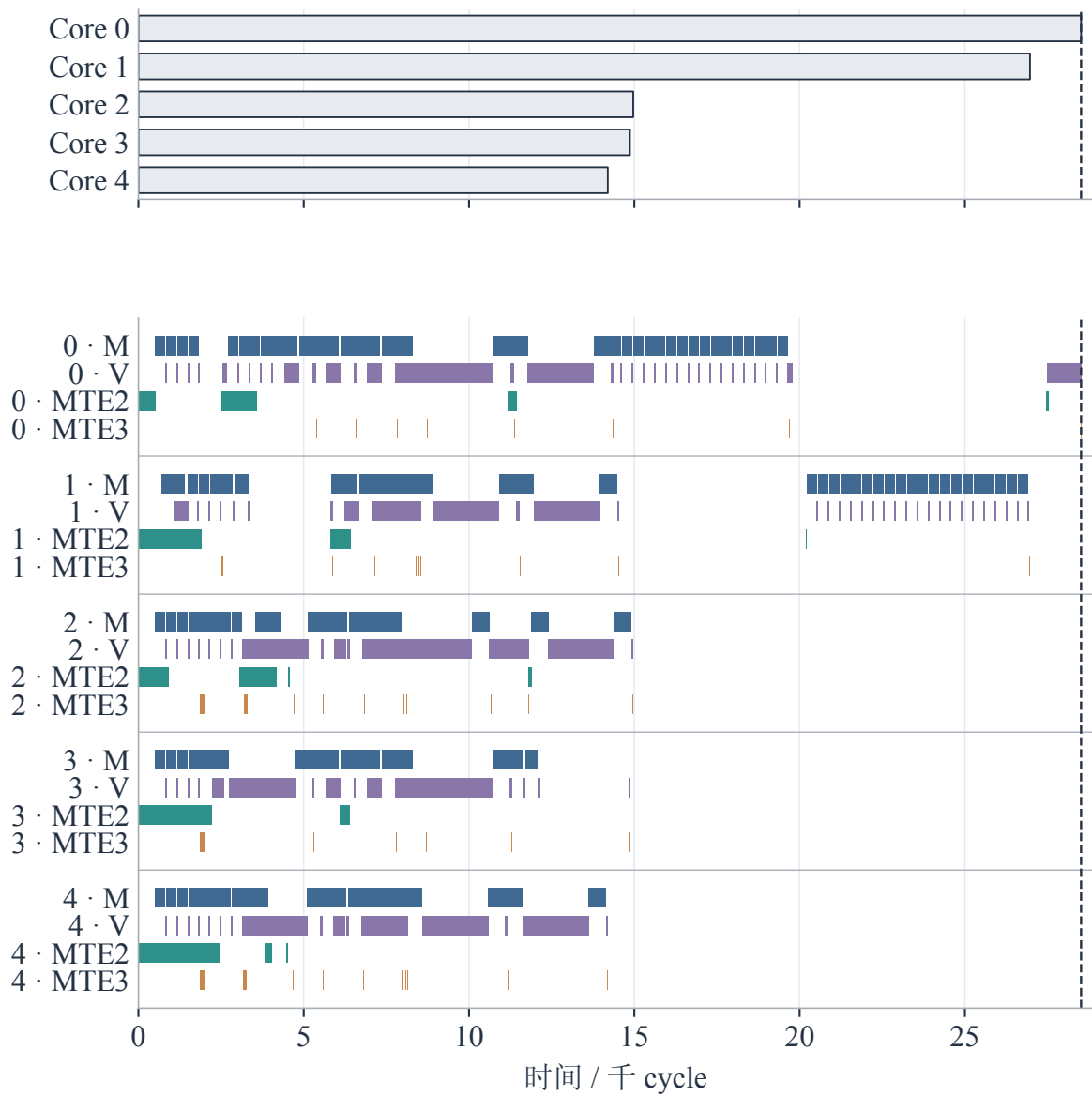


图 5.4 同一用例的 C04 候选，使用相同横轴。搬运量下降且 spill 为零，但 Makespan 增至 28,514 cycles，说明字节代理不足以判定执行收益。

6 问题三：归约森林次序与只读 Cache 协同

6.1 从空间复用到请求次序

问题三保留问题二的同核合并和跨核 COPY 语义，增加所有核心共享的只读 Cache。COPY_IN 发射时按逻辑张量标识查询：命中进入 Cache 读带宽池；未命中进入 DDR 池，传输完成后填入 Cache。命中不改变 FIFO 次序；容量不足时按进入顺序淘汰。

Cache 不消除原数据依赖，不免除源 COPY_OUT 完成后的 500 cycles 跨核等待，也不把命中读变成零时长操作。它改变的是读请求使用的服务资源与后续状态。故 $M_3(P)$ 取决于请求序列、发射时刻、共享输入和在途传输，不能仅用总命中字节乘一个固定节省系数估算。

本文沿用前两问的结构—评价组织方式：首先按图形态构造合法的分核和序列；对共享输入流水、注意力结构中的次序及私有归约森林分别提供有守卫候选；再以固定候选下界排除不可能改善者，最终只接受完整官方评价的严格 Makespan 改善。主算法称为 Forest，指包含森林次序候选的完整固定策略，而不意味着所有输入都是归约树。

例如，case_021、3 Core 的编译后张量 t1000000001 大小为 4096 B。官方事件给出以下生命周期；事件时刻为 cycles，表中间隔不表示等长处理阶段。

表 6.1 同一张量的 Cache 事件

时刻 / cycle	事件
0	COPY_IN 发射时未命中，尚未填充
207	DDR COPY_IN 完成，张量插入 FIFO
4,956	另一 Core 读命中，FIFO 年龄不刷新
323,850	插入张量 t1000000487 时，该最老项被淘汰

6.2 私有归约树的内部存活前沿

归约树的多个子树先产生结果，父操作等子结果齐备后消费。处理子树的先后顺序决定先完成的结果需要保留多久。在固定私有树、每个内部结果唯一生产且只由父结点消费、每结点产生一个已知大小输出的条件下，可以把子树压缩为一对量：内部峰值 h_i 与最终保留结果大小 r_i 。

此处限定非交错执行：一个子树完成后才处理下一个兄弟；只统计内部结果的标量字节；父输出先分配，再释放孩子结果。令兄弟次序为 π ，则峰值为

$$H(\pi) = \max \left\{ \max_j \left(\sum_{\ell < j} r_{\pi_\ell} + h_{\pi_j} \right), \sum_i r_i + r_{\text{parent}} \right\}.$$

第一项表示处理第 j 个子树时，之前兄弟留下的结果与当前子树内部峰值叠加；第二项表示父输出分配瞬间仍须保留所有孩子结果。这个式子给出了“先做哪个分支”的具体数学对象。

命题 6.1 (内部前沿排序)。若 $h_i \geq r_i \geq 0$ ，按 $h_i - r_i$ 非增顺序处理兄弟子树，在上述非交错模型中最小化 H 。

证明。考虑相邻的 A, B ，且 $h_A - r_A \geq h_B - r_B$ 。顺序 A, B 与 B, A 的局部峰值分别为

$$H_{AB} = \max(h_A, r_A + h_B), \quad H_{BA} = \max(h_B, r_B + h_A).$$

由条件 $r_A + h_B \leq r_B + h_A$ ，且 $h_A \leq r_B + h_A$ ，故 $H_{AB} \leq H_{BA}$ 。两种次序处理后均保

留 $r_A + r_B$ ，不会改变后续兄弟的初始存活量；共同前缀只为二者峰值加上同一个常量，父分配项也不变。因此交换逆序相邻对不增加峰值。对兄弟次序重复交换，再对子树自底向上归纳，得到结论。□

这一交换论证与经典表达式树代码生成中的存储次序思想相关 [4]，本文的适配对象是已知字节大小的内部结果。算法据此递推 h, r ，生成深度优先后序，保持原来的归核结果，再用 singleton 子图序列表达。森林识别还要求原始 COPY 只在合法边界出现、内部输出无 fanout、无缺少张量大小的额外直接计算边等。

命题最小化的是内部前沿代理。外部输入、不同归约树之间的根结果保留、L1/UB 双池、编译后物理实例、COPY 和真实存储申请次序不在这个标量模型内。因此它不是全局缓存峰值最优，也不是零 spill 或 Makespan 最优定理。官方评价是将有理论依据的次序转化为机器层结论的必要一步。

取 $h_A = 8, r_A = 2, h_B = 7, r_B = 5$ ，父输出大小为 3（单位均为 KiB）。按 $A \rightarrow B$ 处理时，局部峰值为 $\max\{8, 2 + 7, 2 + 5 + 3\} = 10$ ；反序峰值为 $\max\{7, 5 + 8, 5 + 2 + 3\} = 13$ 。该例保留父输出先分配的费用，只解释内部前沿排序，不是官方存储池的实测轨迹。

6.3 固定候选的下界剪枝与接受规则

在 singleton 计划、Core 归属和同核同 Pipe FIFO 都已经确定时，将有证据的必要计算依赖、FIFO 先后及跨核释放约束构成加权有向图。对每条必要关系 $u \rightarrow v$ 写成

$$s_v \geq s_u + d_u + \delta_{uv},$$

其中 δ_{uv} 只包含已经证明不可避免的等待；同端点多条约束取最大值，不相加。图上最长路径 $L(P)$ 是固定候选任意成功执行的下界。若当前 fresh incumbent 的官方完成时间为 U ，则 $L(P) \geq U$ 时，该候选不可能严格改善，可省去完整 E0。

这里不能使用任意“去掉资源限制后重新贪心调度”的耗时作为下界：资源放松扩大了可行集合，却不保证一个固定贪心策略的输出单调改善。本文的下界来自每条真实执行都满足的不等式与路径，而不是放松模拟器恰好给出的一个解。

算法 3：Forest 完整策略。

输入：G、K、冻结配置

- 1 从当前输入建立结构索引，生成并评价结构/日历基准方案
- 2 对适用结构，考虑共享输入连续分段或保留注意力分配见证的次序
- 3 在剩余总预算内，识别私有归约森林并生成 h-r 排序候选
- 4 对重复计划直接跳过；对满足条件的候选计算固定计划下界
- 5 若下界不可能击败 incumbent，则保留剪枝证据并跳过 E0
- 6 否则作完整官方 P3 评价；只有 Makespan 严格降低才替换
- 7 总在线 E0 不超过 3 次，输出最佳成功方案及完整调用记录

三个候选名额为整次调用共享，不是每个机制各有三次。某些结构的早期候选已使用预算时，后续森林候选不会继续新增评价。结构不适用、重复、下界剪枝、合法性拒绝及实际收益不佳是不同停止原因。

对给定归约森林，识别和后序遍历为 $O(n + m)$ ；兄弟排序为 $O(\sum_v \deg(v) \log \deg(v))$ 。固定候选 DAG 的最长路径为 $O(n + m')$ ，其中 m' 包含必要 FIFO 等边；图不在已证域内时不使用该下界。整体还包含结构索引、分配构造与至多三次官方评价，不能将整个 Forest 的复杂度等同于树排序子过程。

6.4 同计划 Cache 对照与完整算法结果

Forest 固定版本在 100 图 $\times 1 \sim 5$ 核上全部得到官方成功结果，500 次新求解过程共使用 973 次在线 E0，无重试。相对前一完整策略，24 格 Makespan 改善，476 格持平，0 格退化。额外 COPY 与 spill 各有 18 格减少、479 格持平、3 格增加，净变化各为 -413450240 B；spill 是总搬运的组成部分，二者不能相加声称减少了双倍字节。

为了隔离 Cache 作用，对每个最终计划使用相同图、核数与计划分别评价问题二和问题三。这里的无 L2 对照是 P3 计划在 P2 语义下的结果，不是问题二主算法另行优化出的赢家。

核数	无 L2: $\text{mean}(B/M_2)$	Cache: $\text{mean}(B/M_3)$	$\text{mean}(M_2/M_3)$	逐图字节命中率 均值
1	1.1971	1.1998	1.0023	6.47%
2	2.2949	2.3120	1.0079	16.25%
3	3.1745	3.2587	1.0314	25.96%
4	3.9211	4.0915	1.0512	24.53%
5	4.4763	4.7576	1.0829	31.36%

5 核的逐图 Cache 加速比平均为 1.0829，但这不是“所有用例都快了 8.29%”，也不是完成时间百分比下降的平均值。跨图访问字节加权的 5 核命中率为 47.45%，与上表的逐图算术平均 31.36% 含义不同，大流量图在前者中权重更大。

Forest 求解墙钟均值为 4.401 s、中位数 1.616 s、最近秩 P95 为 20.009 s、最大值为 51.335 s。它包括在线 E0，来源是共享主机四 worker 的完整批次。后续 R9F 批次的单 worker 时间不用于替换这些数值。

6.5 选择政策的负消融：相对收益与绝对收益可能冲突

后续 R9F 对 5 核构造增加了一项联合门槛：候选须使 P3 的 M_3 严格降低、同计划 P2 的 M_2 不增加，同时 $G = M_2/M_3$ 不降低。100 个 5 核图中，96 个不满足构造识别条件，1 个由候选下界剪枝，3 个被联合门槛拒绝，最终接受数为零。全 500 个最终方案均与 Forest 字节相同。

三个被拒绝的候选实际上同时降低了无 L2 和 Cache 的绝对完成时间：

case/K	原 M_2	候选 M_2	原 M_3	候选 M_3
064/5	7029	6899	6981	6877
068/5	131631	98913	116345	97971
088/5	97305	66729	85789	66224

联合门槛中的比值条件等价于

$$M_{2,\text{new}}M_{3,\text{old}} \geq M_{2,\text{old}}M_{3,\text{new}}.$$

由于分子随方案改变，无 L2 时间改善得更快时，Cache 相对收益可能下降，即使 M_3 已明显缩短。因此这个负消融表明“维持相对 Cache 收益”的政策可能排除主要目标更优的方案，不能解释成第五核构造完全没有绝对收益。该门槛是研究时采用的选择政策，官方首要目标仍是 Makespan。

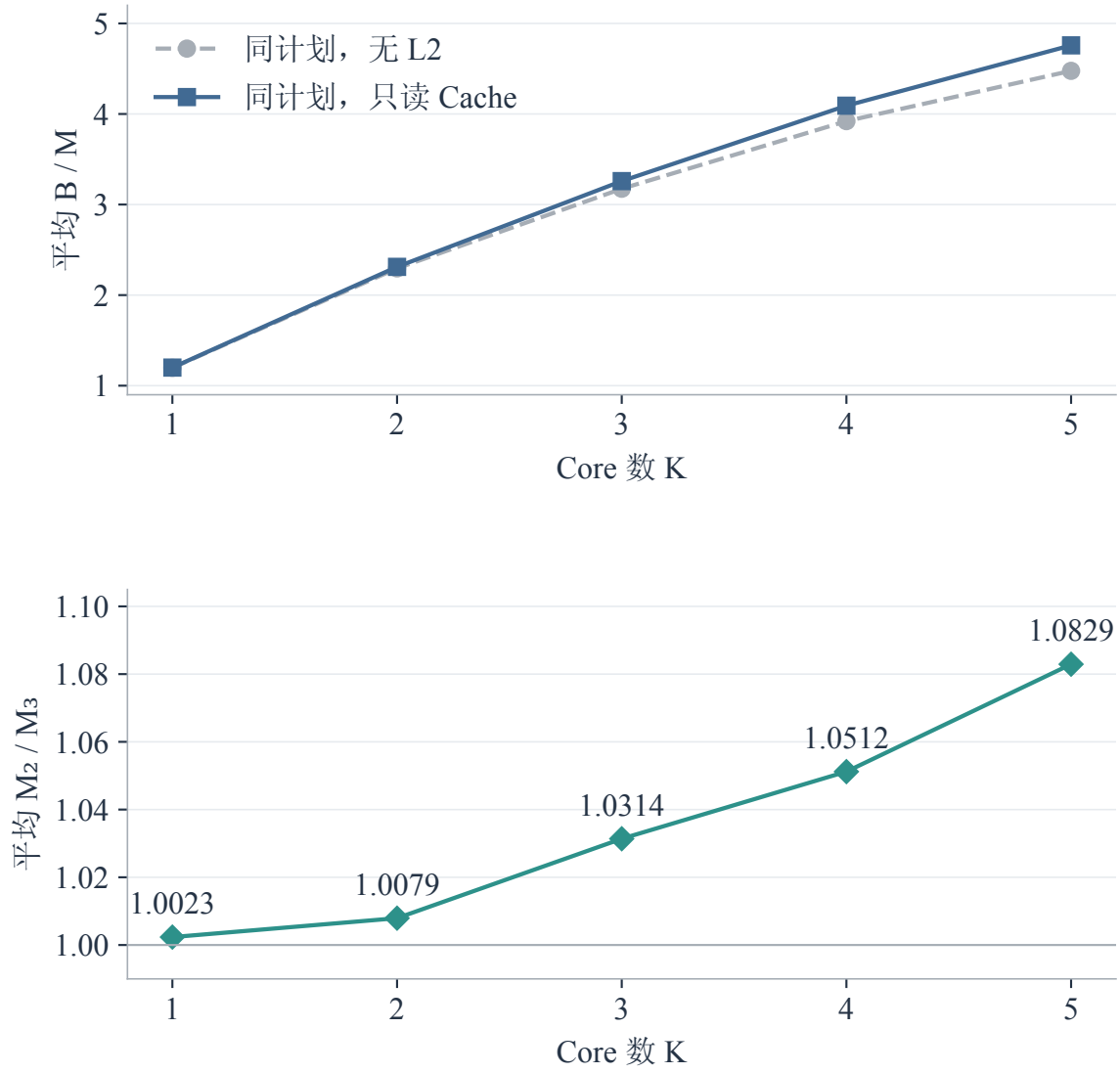


图 6.1 问题三的同计划配对评价。上：无 L2 与只读 Cache 的平均基线加速比；下：逐图 M_2/M_3 的均值。每个核数包含 100 对，不用均值之比替代比值均值。

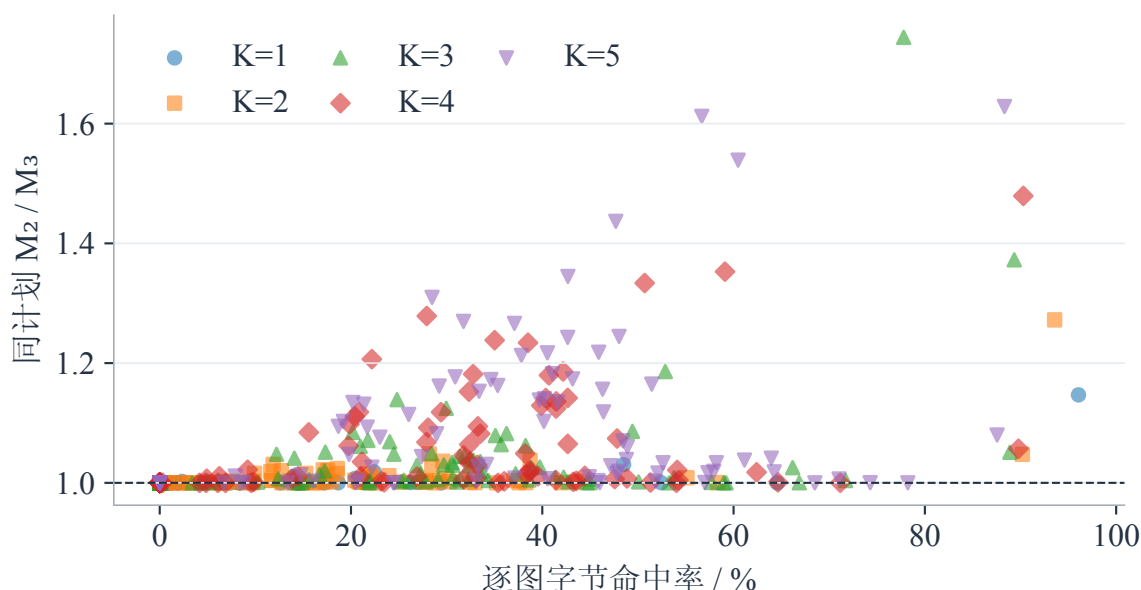


图 6.2 500 个同计划配对结果中的字节命中率与 Cache 收益。保留低于 1 的负收益点；100 个用例的不同核数结果相关，不能当作 500 个独立随机样本。

本文仍以 Forest 的固定全量作为主结果，并明确它不是所有已测单格候选中的最小 M_3 。将三格候选手工替换到主表会变成事后逐格选择，不能代表一个已经完成全量验证的新算法。R9F 的 400+100 两段独立执行共计 982 次在线 E0，只用于说明这项冻结政策的实际行为。

本章把计算树上的可证明排序、候选级必要条件和共享 Cache 的真实观测放在同一条论证链上。最有价值的结论既包括何时能减少内部前沿，也包括这种局部最优性为什么不足以取代最终执行评价。

7 实验设计与综合分析

7.1 数据、版本与对照协议

实验采用赛题给定的全部 100 个原始计算图，核数取 1~5。每题对应 500 格，三题共 1500 格。所有主成绩使用同一题内冻结的完整求解器、统一入口、固定候选规则和预算，不以每个格子的历史最优组合代表一个算法。原始图和配置不清洗、不改写；逐图基线与对应的官方结果绑定。

本文区分三种比较。第一种是官方单核启发式基线与多核方案之比，用于题目要求的加速曲线；第二种是问题、同图、同核数下两个完整固定版本的结果，用于观察方法演进；第三种是相同 P3 方案在无 L2 与只读 Cache 条件下的结果，用于隔离 Cache 作用。三问硬件语义本身不同，不用 P1 与 P2 曲线的差直接估计某一个算法模块的贡献。

正文表格和后续图件使用同一份逐格结果表。导出程序对 100×5 覆盖、单核分母身份及原件字节进行核对，对 P3 另外读取同计划无 L2 原件，重算每核平均值。独立作者审计提供完整计划、官方结果及调用账的对应证据；论文阶段的数据导出是静态复核，不声称重新执行了全部官方评分。

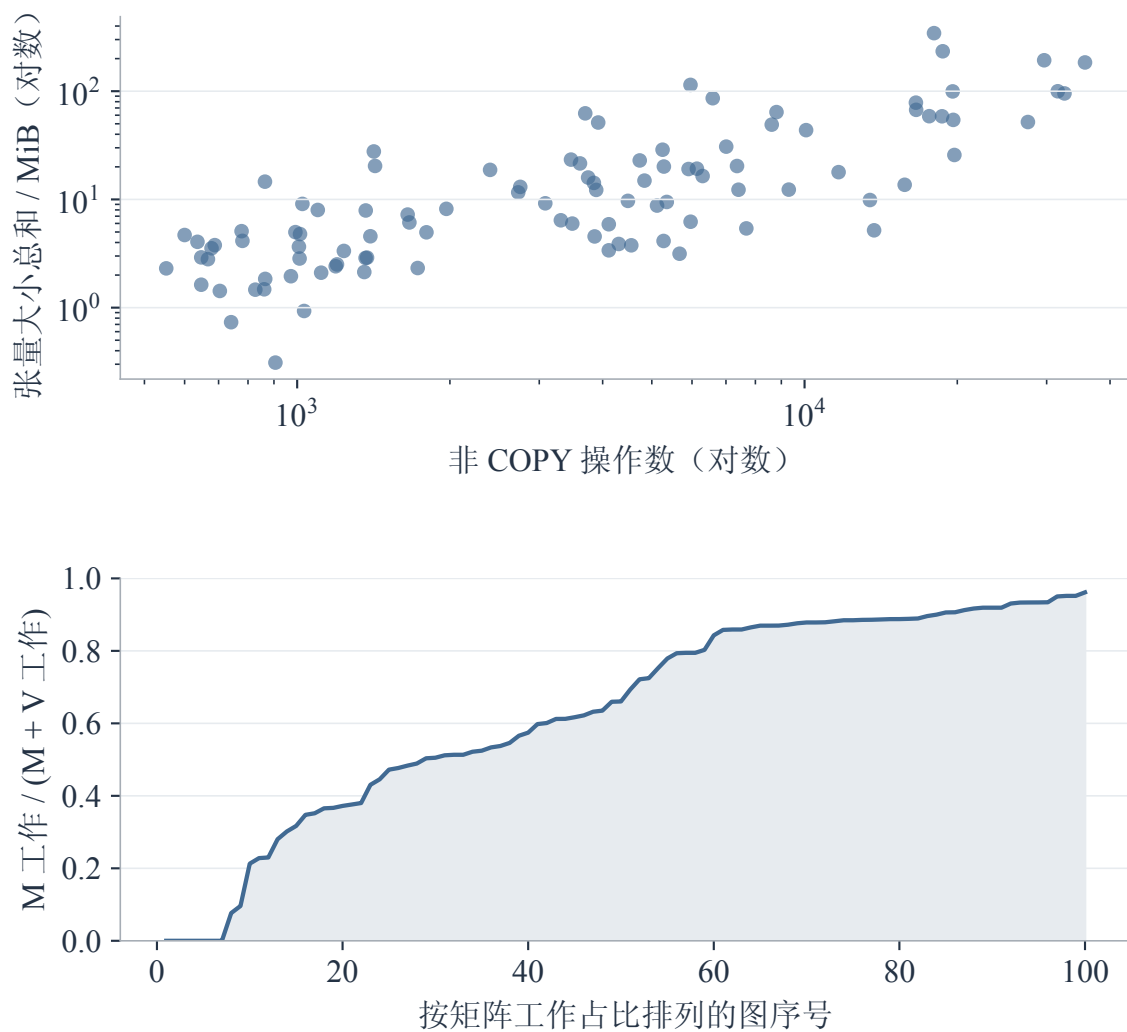


图 7.1 官方 100 张计算图的结构统计。上：非 COPY 操作数与张量大小总和；下：按矩阵 Pipe 工作占比排序的用例分布。

7.2 主要质量结果

下表为各问题优化方案相对共同官方单核分母的逐图比值均值。P1/P2 题目主曲线的 1 核坐标另按规定设为 1；本表保留算法优化单核表现，以揭示基线本身的启发式性质。

核数	P1 优化方案	P2 优化方案	P3 优化方案	P3 同计划无 L2
1	1.0021	1.2061	1.1998	1.1971
2	1.9516	2.3167	2.3120	2.2949
3	2.7514	3.2356	3.2587	3.1745
4	3.4786	3.9711	4.0915	3.9211
5	4.0553	4.5498	4.7576	4.4763

P1 的完整 Task 边界限制了跨子图复用；P2/P3 允许同核合并，但它们仍受共享 DDR、跨核释放和缓存压力影响。表中的结果与这个递进关系相符，却不能单独证明其差异全部由某一项资源解释。机制证据需要回到同条件方案对照及事件轨迹。

此外，P2 主算法与“P3 同计划无 L2”一列是两个不同的方案生成策略。后者只是为 P3 隔离 Cache 作用的对照，不应被解释为 P2 主算法退化。5 核 P3 的 4.7576 与无 L2 的 4.4763 之比约为 1.0628，而逐图 M_2/M_3 的均值为 1.0829；不同聚合顺序产生不同结果，正文以题目要求和配对定义为准。

7.3 求解效率与资源成本

三种方法均把结构预处理、候选构造、在线评分及必要输出包含在求解墙钟中。P1/P2 在线使用加速评价接口，最终 E0 复评独立计时；P3 将官方 E0 直接用于在线选择，其时间已经包含在求解器内。

方法与样本范围	在线评价总次数	求解均值 / s	P95 / s	最大 / s	运行条件
P1, 100 个 5 核格	全 500 格 1130 次 E1	3.522	16.623	25.552	共享主机, 4 worker
P2, 全部 500 格	1131 次 E2	4.029	18.841	40.187	共享主机, 1 worker
P3 Forest, 全部 500 格	973 次 E0	4.401	20.009	51.335	共享主机, 4 worker

P1 作者表的 P95 采用线性插值定义，P2/P3 采用最近秩定义；不同定义在图表中注明，后续统一分布图可从逐格时间重新计算同一口径，不能混用后宣称精细排名。上表 P1 的时间只覆盖 5 核，评价次数覆盖全 500 格，两个范围特意分开标注。

题面建议单个用例在 5~10 分钟内给出方案，上述成功进程的实测时间明显小于这一量级。但实验未进行独占硬件、清空操作系统缓存或跨平台重复测量，故只能报告给定共享环境的测量范围。算法设计仍以同时降低官方 M 和端到端墙钟为目标，不以进入 10 分钟以内作为效率优化的终点。

7.4 机制证据、消融与负结果

三类已有证据承担不同任务。P1/P2/P3 前后完整版本对照展示最终固定策略的整体变化；每个被选中机制的记录说明它确实触发并影响了输出；真实局部结构和时间线解释一

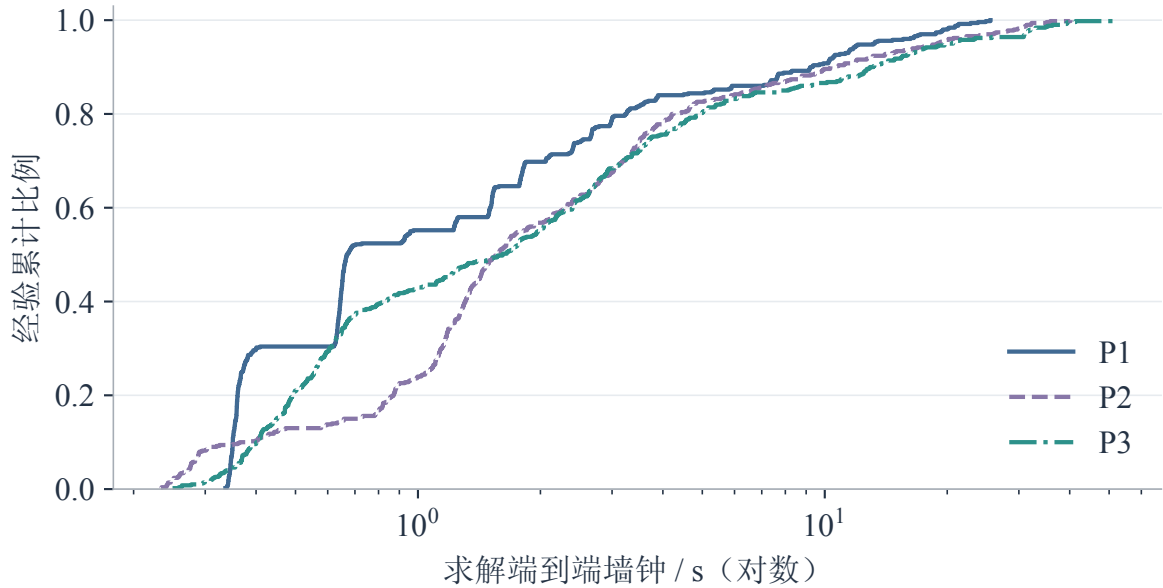


图 7.2 三种完整求解器的端到端墙钟经验分布，每题 500 次。时间含各自在线评价，横轴为对数；共享主机和并发条件不同，不能据此作独占硬件速度排序。

格中发生了什么。若多个机制同时变化，前后版本差不能自动分摊成各模块的独立贡献。

P1 三片援助提供了新增并行机会，也增加了部分边界搬运；5 核最终版本的平均加速比改善约 0.729%，额外 DDR 总量增加约 0.389%。P2 完整新版本在 268 格上缩短 M，但总额外 DDR 增加约 36.66%；这一取舍说明并行与流水收益可以超过新增通信的代价。P3 森林次序带来 24 格 M 改善，却仍存在 3 格额外 COPY 增加，因此没有全指标支配关系。

C04 六个预选格全部退化，否定了对该固定构造的收益预期，却不足以否定整个零 spill 或联合通信构造方向。R9F 三个候选绝对 M 改善但 G 下降，揭示的是接受政策与主要目标的冲突；它们未被完整策略接受，不能手工加入主表。这些负结果使方法选择的依据更清楚，也防止只展示成功图片导致过度归因。

7.5 下界、可证差距与理论未覆盖部分

当必要下界与方案上界来自同一执行域时，可用 $U/L - 1$ 给出相对最优差距的保守上界。P2 已有冻结证书报告：在 1~5 核下，满足 $U \leq 1.05L$ 的图数分别为 81、62、32、33、23。这个统计意味着相应格可在该证书条件下认证距最优不超过 5%；它不是在 500 个互相独立样本上的统计推断。

5 核的平均 B/L 约为 6.108，明显高于实现的 4.550。二者的差既可能来自构造尚有改进空间，也可能来自下界忽略真实通信和容量约束；不能把 6.108 写成已经证实可达的性能目标。附录给出的强化窗口与残余等待组合是数学推导，尚未生成对应的新全量证书，故不替换这里的冻结数值。

7.6 有效性边界

全部 100 个官方图均参与了研发，本文结果属于给定公开评测集合上的完整证据，而非未见分布上的泛化测试。同一图的五个核数相互关联，不能把 500 格直接当作 500 个独立随机样本计算显著性。算法按输入结构而非 case 编号选策略能够减少显式查表，但不消

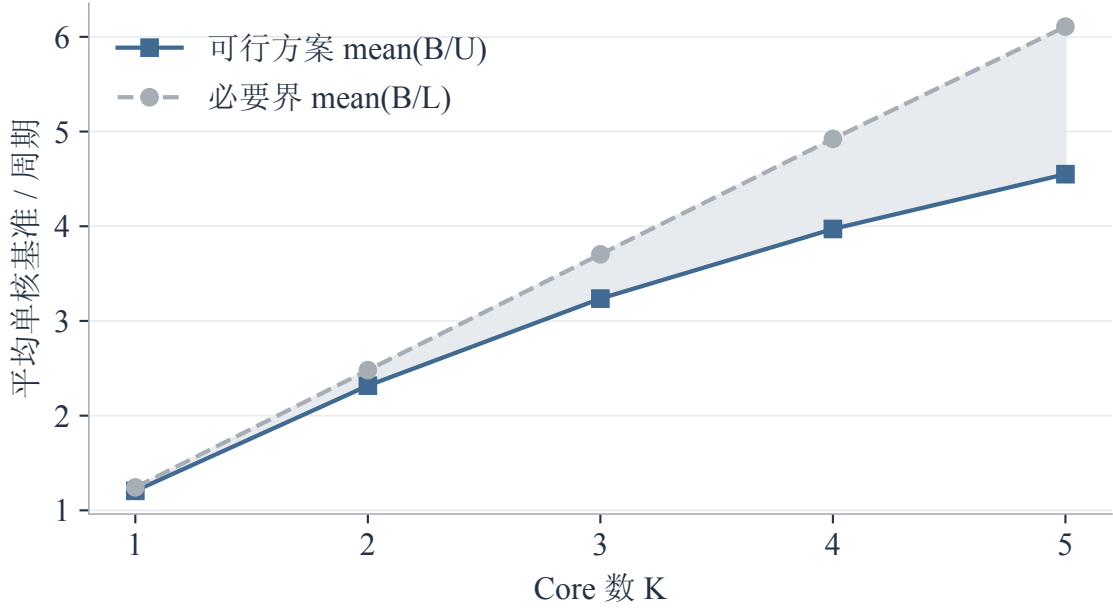


图 7.3 同一优化域中的可行结果与必要界。 $L \leq \text{OPT} \leq U$ ，阴影是必要界给出的上限与当前结果之间的差距，不代表一定可实现的收益。

除设计过程中接触公开图所带来的适配性。

本文没有将零散历史最优合并为主成绩，也没有把某次局部评分循环加速推断为完整求解器相同比例的加速。程序失败、超时与未运行应逐格保留；当前主批次完整成功，并不意味着所有未来输入都不会失败。最后，官方事件模拟是统一比较条件，其结果不等于已在真实 NPU 上验证的端到端收益。

8 模型评价与结论

8.1 方法的优势

本文以同一个可提交接口和执行语义层次连接三问。问题一围绕 Task 边界处理屏障与并行度；问题二利用同核驻留，把张量复用引入分核费用并与日历构造结合；问题三进一步安排归约次序和共享输入访问，在 Cache 语义下重新评价。三种方法共同采用结构守卫、有限候选、必要条件与完整评价，使每一项简化都有明确用途和验证边界。

数学分析直接服务于算法选择。秩函数解释为什么局部援助不制造等待环；超边触核费用解释为什么联合移动可能优于逐点迁移；树前沿交换证明给出无需枚举兄弟排列的次序；候选下界避免为不可能获胜的方案支付完整评价成本。这些结论与代码中实际检查的条件相连，而不是独立放置的理论装饰。

实验保留完整固定版本及逐格原件，区分方案质量与求解速度。真实 JSON、子图映射和事件时间线使用同源标识，使机制解释能够追踪到具体执行。负例与代价同时报告，有助于说明为何不能仅用搬运量、命中率或局部负载衡量算法。

8.2 局限与可推广条件

首先，完整执行仍然具有动态耦合。共享 DDR、Cache 填充/淘汰、FIFO 和物理存储申请会相互影响，本文没有证明一个固定权重 DAG 足以精确表示全部行为，也未构造一

般未来充分状态。局部编译复用、响应表示和完整方案去重应继续分别检查适用条件。

其次，结构识别限制了专用构造的适用范围。私有树、单输出、唯一生产者或无别名条件被破坏后，已有局部最优性不能直接延续；需要重新定义状态、证明或退回一般构造。容量、带宽、Core 数和官方核内策略改变时，也需重新校验费用模型、候选界和固定预算下的表现。

再次，当前主策略仍有可改进空间。R9F 的比值门槛排除了绝对 Makespan 更好的局部候选，说明辅助指标应服务主要目标；P2 中通信量与时间的取舍尚不能完全由简洁代理解释；P1 中细粒度并行仍可能受大额边界延迟限制。对这些问题，更合理的后续工作是围绕具体反例修正构造和接受规则，再为新的固定算法建立完整证据，而不是事后拼接局部最佳数值。

最后，本文的数据来自给定的公开图与官方模拟，尚无未见分布、不同硬件条件和真实设备的效果验证。推广需要相应的新证据，不能从当前完整评测直接推出跨平台收益。

8.3 结论

针对多核 NPU 的三种执行场景，本文形成了语义一致的结构化切分与调度方法。问题一通过屏障感知候选与有序证书的支路援助组织并行，问题二以复用超图和空隙插入联合处理通信及计算，问题三将私有归约树的存储次序与官方 Cache 执行闭环结合。三个固定算法均完成 100 图 $\times 1 \sim 5$ 核的完整评测，5 核平均基线加速比分别为 4.0553、4.5498 和 4.7576；P3 同计划 Cache 加速比均值为 1.0829。

结果支持本文的核心判断：图上的结构改动只有经过相应机器语义的解释，才能确定实际收益。有效的数学简化应保留当前决策所需的信息，并清楚声明不能推断的部分。由此形成的统一研究框架，将定义、局部证明、可执行构造、真实流水和完整实验连接为一条可检查的论证链。

参考文献

- [1] 第二十三届中国研究生数学建模竞赛. A 题：通用神经网络处理器下的多核调度问题及官方附件 [Z]. 2026.
- [2] GRAHAM R L. Bounds for Certain Multiprocessing Anomalies[J]. Bell System Technical Journal, 1966, 45(9): 1563–1581. DOI: 10.1002/j.1538-7305.1966.tb01709.x.
- [3] ÇATALYÜREK Ü V, AYKANAT C. Hypergraph-Partitioning-Based Decomposition for Parallel Sparse-Matrix Vector Multiplication[J]. IEEE Transactions on Parallel and Distributed Systems, 1999, 10(7): 673–693. DOI: 10.1109/71.780863.
- [4] SETHI R, ULLMAN J D. The Generation of Optimal Code for Arithmetic Expressions[J]. Journal of the ACM, 1970, 17(4): 715–728. DOI: 10.1145/321607.321620.

附录 A 补充数学推导

A.1 成功执行的必要计算投影

在唯一原生产者、无原始逻辑身份混叠等保守条件下，只保留原计算之间直接存在的依赖，及同一张量原生产者到消费者的关系，不随意收缩经过原 COPY 的路径。任意官方成功执行保留全部原始计算及其时长，每核每 Pipe 仍为单槽。取真实轨迹中这些计算的原核及起止时刻，得到投影图的一个可行调度。

跨 Task 或跨核的因果不能只看一个被删除的逻辑边。P1 通过前驱 Task 完成与激活门控保留因果；P2/P3 通过源 OUT、基础 IN 和 500 周期释放保留因果。接收侧后续 reload 还受 Step2 顺序及 MTE2 FIFO 制约，不可越过必要的基础 IN。该投影论证说明保留的约束对每条成功轨迹成立；它不是把真实约束删掉后重新运行一个贪心调度器。

A.2 非空窗口中的工作量与不可分原子

在必要计算投影上，定义不含自身的最长前缀和后缀

$$h_v = \max_{u \rightarrow v} (h_u + d_u), \quad t_v = \max_{v \rightarrow w} (d_w + t_w).$$

取非负 a, b 及某一 Pipe 的非空集合

$$S = \{v : p_v = p, h_v \geq a, t_v \geq b\}.$$

这些操作在任何成功执行中完整落在 $[a, M - b]$ 内。按时长降序记 $d_{(1)} \geq \dots \geq d_{(n)}$ ，令

$$A_K(S) = \max_{\substack{r \geq 1 \\ m=(r-1)K+1 \leq n}} \sum_{j=m-r+1}^m d_{(j)}.$$

则有

$$M \geq a + b + \max \left\{ \left\lceil \frac{\sum_{v \in S} d_v}{K} \right\rceil, A_K(S) \right\}.$$

证明。窗口长度为 $x = M - a - b$ ，同 Pipe 的 K 个槽总工作容量为 Kx ，给出工作量项。对前 $m = (r-1)K + 1$ 个操作，由鸽巢原理至少一个 Core 分到 r 个；即使选取其中最长的 r 个，该核心也必须占用 $\sum_{j=m-r+1}^m d_{(j)}$ 。逐 r 取最大得到原子项。两者是同一窗口的必要条件，取最大值而不是相加。最后只加一次头尾。□

例如两个 Core 上三个不可分操作时长为 10、8、7，工作量给出 13，简单均匀原子界可能只给 14，而上述界给出 15。这个例子只解释不可分性；是否能在真实机器达到 15，还取决于其他依赖和资源。

计算 head/tail 为 $O(n + m)$ ；给定窗口后，成员检查与时长排序分别为 $O(n)$ 及 $O(|S| \log |S|)$ 。这不意味着遍历所有双阈值窗口也具有同样复杂度。空集合不能贡献 $a + b$ 。

A.3 通用计算屏障与残余等待

取一条有序的通用计算屏障链；“通用”指这些计算与相关其他计算均由有向路径可比较，使屏障间的计算能够定位在互不重叠的实际时间块中。跨核必要等待记为 τ_q ，P1 为 1000，P2/P3 为 500。该常数仅用于已有因果证明的跨核计算路径，不把可提前预读的输入事件强塞到计算块中。

对块内一个非空同 Pipe 窗口，头尾阈值为 a, b ，左、右是否存在屏障分别为 $\ell, r \in \{0, 1\}$ 。

头尾路径可能已经覆盖部分跨核等待，尚需保留的等待仅为

$$c(a, b) = \ell(\tau_q - a)_+ + r(\tau_q - b)_+.$$

设块长为 x ，窗口长 $z = x - a - b$ ，安全的容量上包络为

$$W(S) \leq z + (K - 1)(z - c)_+.$$

两端屏障同核时，一个 Core 可利用全部 z ，其他核至多利用 $(z - c)_+$ 。两端异核时，令 $A = (\tau_q - a)_+$ 、 $B = (\tau_q - b)_+$ ，总容量至多

$$(z - A)_+ + (z - B)_+ + (K - 2)(z - A - B)_+.$$

由对 $z \leq \min(A, B)$ 、中间区间及 $z \geq A + B$ 分段，可验证

$$(z - A)_+ + (z - B)_+ \leq z + (z - A - B)_+,$$

故仍受相同上包络约束。整数容量不等式 $W \leq z + (K - 1)(z - c)_+$ 的最小可行长度为

$$\phi_{K,c}(W) = \min \left\{ W, \left\lceil \frac{W + (K - 1)c}{K} \right\rceil \right\}.$$

当 $W \leq c$ 时只用首核的 $z = W$ 即可满足放松容量；当 $W > c$ 时解第二段线性不等式并取整。结合 B.2 原子项，得到

$$x \geq a + b + \max\{\phi_{K,c(a,b)}(W(S)), A_K(S)\}.$$

各块与屏障区间互不重叠，故可将每块已证下界与屏障计算时间相加。仅仅从若干互相重叠窗口取得下界时不能相加，应取最大值。P1 的同核 100 周期等待也不能任意加进来，因为一个计算屏障并不强迫方案在此处切开 Task。

本式的要点是扣除头尾已覆盖的等待。把 $a + b$ 、完整双向等待与工作量简单全部相加，会重复计费。该强化组合用于理论分析；本文主表仍引用既有冻结证书，没有声称已为新公式重新生成 500 格数值。

A.4 未来充分性与当前评分表示

第 3 章给出的三条件说明，动作是否可用、压缩状态怎样更新和当前观测怎样恢复，都必须由压缩状态决定。只满足第三项是当前评分表示；保存一个模拟器的运行现场是 runtime checkpoint；缓存同一局部编译产物是编译复用。它们各自可以有效节省计算，但不自动满足关于所有未来动作词的等价要求。

固定观测也很重要。若完整结果能恢复计划标签与序列，要求“完整结果相同”可能让等价类极细，失去压缩收益；若只保留 Makespan，当前等价类又可能在后续切分或移动后分裂。研究应从实际想省掉的工作和需要保留的观测出发，而不预设必须存在很小的全局状态空间。

附录 B 固定实现与复现信息

三题固定求解器的提交分别为下列 SHA。逐格图、计划、结果来源及完整 SHA-256 见配套 CSV 与来源收据。固定版本结果不能用不同算法的历史赢家拼接。

- P1: 834d8c957538ee069c66aadac9509552a4cc69d7; 在线评分至多 10 次 E1。
- P2: c66559a6f8a31ef7b4720e1f7c3c28d61f8dff3f; 在线评分至多 3 次 E2。
- P3: 311322b996c0948e8a6a9c7ec6ddfe6ae41fbee1; 在线评分至多 3 次 E0。

官方代码与配置身份分别为 de11a83db8d7c47ed328b15a7df71d613a833b16cd23ee9fe877999578a1ace0 和 dcd10de54b23f8366428fb24e828812b1da9549e6eae4a3c3f38604fe5ae77b9。来源收据记录 822 个固定 Git 对象；本稿的数据导出仅读取已有原件，没有新增求解器或评价器调用。